EE449 MAJOR PROJECT REPORT

On

DESIGN AND IMPLEMENTATION OF A VLSI-BASED LOW-POWER 8-BIT ALU USING REVERSIBLE LOGIC GATES

Under the guidance of

Prof. K P Vittal

Submitted in partial fulfilment of the requirements for the degree

Of

BACHELOR OF TECHNOLOGY

In

ELECTRICAL AND ELECTRONICS

By

ANIRUDH HARIHARAN
221EE206
AADITYA SUTHAR
221EE101
SARANSH SHRIVASTAVA
221EE146
CHAITANY SATYA
221EE147
VII Sem B.Tech.(EE)

DEPARTMENT OF ELECTRICAL AND ELECTRONIC
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALORE -575025

November, 2025

DECLARATION

We hereby declare that the Report of the U.G. Project Work entitled

"_____

_____

_____"

which is being submitted to the **National Institute of Technology Karnataka, Surathkal**, in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology in _____

In the Department of _____,

this is a bona fide report of the work carried out by me. The material contained in this report has not been submitted to any University or Institution for the award of any degree.

_____

(Register Number, Name & Signature of the Student)

Department of _____

Place: NITK, Surathkal

Date: _____

Signature of the Students

1.

2.

3.

4.

CERTIFICATE

This is to certify that the **U.G. Project Work Report** entitled
"_____
_____"
submitted jointly by

1. _____ (Register No. _____)
2. _____ (Register No. _____)
3. _____ (Register No. _____)
4. _____ (Register No. _____)

as the record of the work carried out by them, is accepted as the **U.G. Project Work Report submission** in partial fulfilment of the requirements for the award of the degree of
Bachelor of Technology in _____
in the Department of _____,
National Institute of Technology Karnataka, Surathkal.

External Guide (if any) **Internal Guide**

(Name and Signature with Date and Seal) (Name and Signature with Date and Seal)

Head of the Department

(Name and Signature with Date and Seal)
Place: NITK, Surathkal
Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to the **Department of Electrical and Electronics Engineering, National Institute of Technology Karnataka, Surathkal**, for providing us with the opportunity to undertake this significant project titled *"***Design and Implementation of a VLSI-Based Low-Power 8-Bit ALU Using Reversible Logic Gates**.*"*

We want to extend our heartfelt thanks to our **guide, Dr. K P Vittal Sir**, for their valuable guidance, encouragement, and continuous support throughout the course of this project. Their insightful suggestions and constant motivation were instrumental in helping us complete our work.

Finally, we would like to thank our parents, friends, and classmates for their encouragement and moral support throughout this project.

ABSTRACT

With the rapid development of computer processors, it is evident that the next stage of the computational paradigm will be quantum technology. From simulating to solving certain classes of complex mathematical equations, quantum computers have demonstrated their power, while being energy efficient and offering exponential speedups in calculations that would take decades even for a supercomputer. The fundamental requirement of a quantum system is reversibility, where every computed logic can be uniquely inverted. This principle directly aligns with reversible logic, which serves as the classical foundation for designing quantum circuits.

This project presents the **design and implementation of a quantum-inspired 8-bit Arithmetic Logic Unit (ALU)** using **reversible logic gates**. The proposed ALU performs basic arithmetic and logical operations—such as **addition, subtraction, multiplication, and comparison**—using quantum-compatible reversible gates, including **Feynman, Toffoli, Peres**, and **Fredkin gates**. Each operation is optimized to minimize garbage outputs, with constant inputs ensuring future compatibility with quantum hardware.

The design is implemented and simulated in **Cadence Virtuoso** to validate its functionality and performance. Each reversible gate—such as the **Feynman**, **Toffoli**, **Peres**, and **Fredkin** gate—has been individually designed and implemented in **Cadence Virtuoso**, ensuring logical and electrical reversibility at the transistor level. Analysing waveforms and voltage outputs validated the design of each gate, confirming stability, correct logic behaviour, and minimal energy loss.

Subsequently, the verified gates were integrated to realize a complete reversible ALU capable of performing arithmetic and logic operations while conserving energy and maintaining information integrity. Simulation results confirm that the proposed architecture achieves **low-power operation and stable switching behaviour**, establishing reversible logic as a promising bridge between **classical CMOS VLSI** and **quantum computing hardware design**.

Keywords: Reversible Logic, Low-Power VLSI, Quantum Computing, 8-bit ALU, Cadence Virtuoso, Feynman Gate, Toffoli Gate, Fredkin Gate, Peres Gate, DKG Gate, TR Gate, BJN Gate, XOR, Inverter, Pass transistor logic, Transmission Gate, Quantum Cost, Energy Efficiency, Reversible Circuits, Logic Reversibility, Waveform Analysis, Voltage Stability, Garbage Output Minimization, Ancilla,

Contents

NOMENCLATURE.....9
CHAPTER – 1 INTRODUCTION.....12
1.1 Background and Motivation.....12
1.2 Reversible Logic: Concept and Significance.....13
1.3 Evolution and Applications of Reversible Logic.....13
1.4 Role of ALU in Digital Systems.....14
1.5 Project Objective and Scope.....15
1.6 Theoretical Foundation: Landauer's and Bennett's Principles.....15
1.7 Reversible ALU: Design Philosophy.....16
1.8 Significance and Applications.....16
CHAPTER – 2 Literature Review and Theoretical Background.....18
2.1 Review of Conventional (Irreversible) ALUs.....18
2.1.1 Overview of Conventional ALUs.....18
2.1.2 Energy Dissipation in Irreversible Logic.....18
2.1.3 Mathematical Representation of Conventional 8-bit ALU.....19
2.1.4 Energy Efficiency Analysis.....21
2.2 Evolution of Reversible Logic Gates.....21
2.2.1 Historical Background.....21
2.2.2 Concept of Reversibility.....22
2.3 Comparative Study of Existing Reversible ALUs.....23
2.3.1 Overview.....23
2.3.2 Design Parameters for Comparison.....23
2.3.3 Review of Early Reversible ALU Architectures.....24
2.4 Basic Reversible Logic Gates (Feynman, Fredkin, Toffoli, Peres, etc.)..26
2.4.1 The Feynman Gate (Controlled-NOT or CNOT Gate).....26
2.4.2 The Fredkin Gate.....27
2.4.3 The Toffoli Gate.....28
2.4.4 The Peres Gate.....28
2.5 Advanced Reversible Gates (DKG, TR, BJN, PPG).....29
2.5.1 DKG Gate (D. K. Gate).....29

2.5.2 TR Gate (Thapliyal–Ranganathan Gate) 30
2.5.3 BJN Gate 30
2.5.4 PPG Gate (Partial Product Generator Gate) 31
2.6 Key Design Parameters 31
Quantum Cost (QC) 31
Garbage Outputs (GO) 32
Constant Inputs (CI) 32
Delay (D) 32
Hardware Complexity (HC) 32
Power Dissipation (P) 33
CHAPTER – 3 Methodology and Design 34
3.1 Design Flow Overview 34
3.2 Schematics of the gates on Cadence 37
3.2.1 The Feynman Gate 37
3.2.2 Toffoli Gate: 37
3.2.3 Fredkin Gate: 38
3.2.4 Peres Gate: 39
3.2.5 DKG Gate: 39
3.2.6 TR Gate: 40
3.2.7 BJN Gate: 41
3.3 Design of 1-bit and 8-bit Reversible Adders (DKG-based) 42
3.4 Design of 1-bit and 8-bit Subtractors 45
3.5 Design of 2-Bit Reversible Multiplier (Peres Gate-based) 48
3.6 Design of Reversible Comparator (TR + BJN Gates) 51
3.7 Implementation Details (Tools Used: Cadence) 54
3.8 Design Constraints (Technology Node: 180 nm) 55
CHAPTER 4 – Simulation, Results, and Analysis 56
4.1 Simulation Setup in Cadence 56
4.2 Basic Gates Waveforms 59
4.2.1 Feynman Gate 59
4.2.2 Toffoli Gate 59

4.2.3 Fredkin Gate 60
4.2.4 Peres Gate 60
4.3 ALU Waveforms 61
4.3.1 8-bit Adder 61
4.3.2 8-bit Subtractor 63
4.3.3 2–bit Multiplier 65
4.4 Discussion of Results 66
CHAPTER 5 – Conclusion and Future Scope 68
5.1 Summary of Work 68
5.2 Key Observations and Achievements 69
5.3 Limitations of the Current Design 70
5.4 Future Enhancements 72
5.5 Final Remarks 73
References 75

NOMENCLATURE

The following terms, symbols, and abbreviations are used throughout this report.

Symbol / Term	Description
A, B, C, D	Input variables of reversible logic gates
P, Q, R, S	Output variables of reversible logic gates
G, E, L	Comparator outputs: Greater, Equal, Less
Cin / Bin	Carry-in / Borrow-in input signal
Cout / Bout	Carry-out / Borrow-out output signal
Sum (Σ)	Arithmetic sum output of the adder
Diff (Δ)	Arithmetic difference output of the subtractor
P_0, P_1, P_2, P_3	Product bits of the 2-bit multiplier
tpd	Propagation delay between input and output transitions
VDD	Supply voltage (1.8 V for 180 nm CMOS)
I(VDD)	Current drawn from the power supply
Pavg	Average power dissipation (µW)
Eop	Energy per operation (pJ)
kT ln(2)	Minimum energy loss per bit of information (Landauer's principle)
f	Operating frequency of input signal (Hz)
V(t)	Instantaneous voltage waveform
I(t)	Instantaneous current waveform
Q = CV	Charge stored on a capacitor (C = capacitance, V = voltage)
P = VI	Instantaneous power relation
E = ∫P(t)dt	Total energy consumed over time

Cload	Load capacitance at output node
td	Delay time of output transition
ΔV / Δt	Slew rate of signal transition
Reversibility	Property where outputs uniquely determine inputs
Garbage Output	Unused output required to maintain reversibility
Ancilla Input	Constant input used to achieve reversibility
Quantum Cost (QC)	Measure of circuit complexity in terms of quantum gates
Logic Depth	Number of cascaded reversible gate levels between input and output
Transistor Count (T)	Number of transistors required per circuit
180 nm CMOS	Technology node used for implementation
ADE	Analog Design Environment (Cadence simulation tool)
ViVA	Virtuoso Visualization and Analysis tool (waveform viewer)
DRC	Design Rule Check – ensures layout compliance with fabrication rules
LVS	Layout Versus Schematic check – verifies schematic and layout equivalence
PDK	Process Design Kit – technology file used for transistor modeling
CPL	Complementary Pass Transistor Logic used for gate design
DKG Gate	4×4 reversible gate used for adders and subtractors
Peres Gate	3×3 reversible gate performing combined AND and XOR operations

TR Gate	Thapliyal–Ranganathan reversible gate used in comparators
BJN Gate	Bruce–Jha–Nayeem reversible gate used for comparator decision logic
ALU	Arithmetic Logic Unit
VLSI	Very Large Scale Integration
CMOS	Complementary Metal–Oxide–Semiconductor
$\mathbf{P_{avg} = (1/T) \int_0^t V(t) \cdot I(t)\, dt}$	Expression used to calculate the average power from the simulation
$\mathbf{\Delta t_p = t_o(50\%) - t_i(50\%)}$	Formula for propagation delay
η = (E_irrev – E_rev) / E_irrev × 100%	Efficiency improvement of the reversible circuit over the irreversible design

CHAPTER – 1 INTRODUCTION

1.1 Background and Motivation

In the modern era of computing and electronics, the demand for **high-performance and energy-efficient systems** is increasing at an unprecedented rate. With the exponential growth of portable devices, embedded systems, and high-density VLSI chips, power consumption has become one of the most critical design parameters in digital electronics. The limitations of **conventional CMOS technology**, particularly in terms of heat dissipation and information loss, have driven researchers to explore **alternative computational paradigms**. One such promising approach is **reversible logic design**.

According to **Landauer's principle**, each bit of information lost results in an energy dissipation of **kTln(2)** joules, where k is Boltzmann's constant and T is the absolute temperature. In traditional logic circuits, when information is lost (for example, when a bit of information is erased), this theoretical limit manifests as heat generation. With the continuously shrinking feature sizes in semiconductor devices, even this seemingly small energy loss per bit becomes significant, leading to **thermal issues, power inefficiency, and reduced device reliability**. [1][4]

Reversible logic offers a robust solution by establishing a one-to-one correspondence between inputs and outputs, ensuring that no information is lost or destroyed during computation. This not only prevents the energy loss associated with bit erasure but also allows for the design of ultra-low-power and quantum-compatible circuits.

To overcome this fundamental limitation, **reversible logic** was introduced. In reversible computation, there exists a **one-to-one correspondence between input and output vectors**, meaning no information is lost during computation. Each unique input produces a unique output, and vice versa. This property enables theoretical **zero-energy computation**, which forms the foundation for **quantum computing**, **low-power VLSI**, and **nanotechnology-based systems**.[2][3][5]

1.2 Reversible Logic: Concept and Significance

Reversible logic circuits differ from traditional digital circuits in several ways:

1. **Information Preservation:**
   Every operation in a reversible circuit is bijective, meaning inputs can always be uniquely recovered from the outputs.
2. **No Fan-Out or Feedback:**
   Reversible circuits prevent direct signal duplication and feedback loops, requiring specialized gates (such as the Feynman gate) for copying signals.
3. **Garbage Outputs and Constant Inputs:**
   To ensure reversibility, extra outputs (called garbage) and fixed inputs (ancilla bits) are added. Optimizing these is crucial for efficient design.
4. **Quantum Cost:**
   As reversible logic is fundamental to quantum computing, each reversible gate has an associated quantum cost, indicating the number of basic quantum operations (1×1 or 2×2 gates) needed for implementation.

The main benefit of reversible logic is energy conservation. By maintaining logical reversibility, the theoretical energy dissipation limit can be significantly lowered, making it ideal for low-power arithmetic units, cryptographic processors, and quantum algorithms.[1][2][3]

1.3 Evolution and Applications of Reversible Logic

This project centres on the **design and implementation of an 8-bit Low-Power Arithmetic Logic Unit (ALU) based on reversible logic principles**. The proposed ALU is built using various **reversible gates**, **including Feynman, Fredkin, Toffoli, Peres, DKG, TR, BJN, and PPG gates**. [6][7][8]

The foundation of reversible logic design can be traced back to the early work of Charles Bennett (1973), who demonstrated that computation can, in principle, be carried out in a thermodynamically reversible manner, provided that no information is destroyed. Since then, various reversible logic gates have been proposed, including:

1. **Feynman Gate (CNOT):** Used for signal duplication and bit inversion.
2. **Fredkin Gate:** A controlled swap gate for data routing**.**

3. **Toffoli Gate (CCNOT):** A universal reversible gate capable of implementing any Boolean function.
4. **Peres Gate:** Combines XOR and AND operations efficiently with low quantum cost.
5. **TR, DKG, BJN, and PPG Gates:** Advanced gates optimized for arithmetic circuits with reduced garbage outputs.

Reversible logic has found applications in:

1. **Quantum computing**, where logical reversibility is a fundamental requirement.
2. **Low-power CMOS design**, which helps minimize energy dissipation.
3. **Optical and DNA computing**, where reversibility ensures high reliability and minimal signal degradation.
4. **Nanotechnology and spintronics**, where reversible principles improve stability and control.

1.4 Role of ALU in Digital Systems

An **Arithmetic Logic Unit (ALU)** is the computational core of any digital system. It performs basic arithmetic operations like addition, subtraction, and multiplication, as well as logical operations such as AND, OR, and XOR. The ALU directly influences a processor's **speed, power consumption, and complexity**, making it a vital candidate for low-power optimization.

Traditional CMOS-based ALUs suffer from **energy loss due to the irreversible nature of their operations, which limits** their use in ultra-low-power and quantum domains. Designing an **ALU using reversible logic** allows the same computational functions to be achieved with **negligible energy dissipation** while preserving input information.[6][7][13]

This project aims to design and implement an **8-bit low-power ALU using reversible logic**, which serves as a proof of concept for next-generation low-power VLSI and quantum architectures.

1.5 Project Objective and Scope

The primary objective of this work is to **design and implement an 8-bit Arithmetic Logic Unit using reversible logic gates** such as Feynman, Fredkin, Toffoli, Peres, DKG, TR, BJN, and PPG. The design focuses on optimizing the following parameters:

1. **Quantum Cost:**
   Minimize the number of basic reversible operations required for each arithmetic function.
2. **Garbage Outputs:**
   Reduce the number of unnecessary outputs generated to maintain reversibility.
3. **Constant Inputs:**
   Limit the use of ancillary bits to enhance circuit efficiency.
4. **Propagation Delay and Power Dissipation:**
   Achieve improved performance with minimal delay and near-zero energy loss.

The **functional blocks** of the proposed reversible ALU include:

1. **Adder/Subtractor unit** using cascaded DKG gates.
2. **Comparator unit** designed with TR and BJN gates.
3. **Multiplier unit** constructed with Peres gates for efficient generation of partial products.
4. **Control unit** for operation selection using reversible switching logic.

The design is **simulated and validated** using Verilog HDL, and performance metrics, including gate count, quantum cost, and garbage outputs, are analyzed.

1.6 Theoretical Foundation: Landauer's and Bennett's Principles

Landauer (1961) demonstrated that erasing one bit of information in any irreversible logic operation necessarily dissipates energy, expressed as:

$$E_{min} = kT\ln 2$$

Bennett (1973) later established that if a computation is made **reversible**, this energy dissipation can be theoretically avoided. Hence, by ensuring every computational step preserves information, **no heat is generated due to logical operations**. This concept is the theoretical backbone of all reversible and quantum computing systems.[1][4]

In this context, reversible logic serves as the bridge between **classical CMOS technology** and **quantum computation**, offering a pathway to practical, low-power hardware realizations.

1.7 Reversible ALU: Design Philosophy

The reversible ALU designed in this work uses a modular architecture, where each arithmetic and logic function is built as an independent reversible block. The DKG-based full adder forms the backbone of arithmetic operations, ensuring minimal quantum cost. Similarly, Peres gates are used in the multiplier for efficient partial product generation, while TR and BJN gates enable compact comparator structures. Each module is interconnected using reversible signal paths that preserve input information. The control unit is also implemented reversibly, ensuring that the entire ALU adheres to the principles of information conservation.[10][12][18]

1.8 Significance and Applications

The proposed design has potential applications in the following areas:

1. Quantum processors: Implementing arithmetic functions in quantum circuits.

2. Low-power embedded systems: Creating ultra-efficient processors for IoT and edge devices.

3. Nanocomputing: Combining with reversible nanostructures for future transistor-less systems.

4. Secure computation: Reversible circuits reduce data loss, making them useful in cryptographic architectures.

Additionally, reversible ALUs pave the way for sustainable electronics, supporting the global goal of decreasing energy consumption in computing devices.

CHAPTER – 2 Literature Review and Theoretical Background

2.1 Review of Conventional (Irreversible) ALUs

2.1.1 Overview of Conventional ALUs

An **Arithmetic Logic Unit (ALU)** is the computational core of every microprocessor and digital system. Conventional ALUs, designed using **Complementary Metal-Oxide Semiconductor (CMOS)** technology, perform arithmetic and logical operations such as addition, subtraction, multiplication, comparison, and bitwise functions.

Traditional CMOS logic employs **irreversible gates**, including AND, OR, NAND, NOR, and XOR. In these gates, the number of inputs and outputs is not equal — for example, an AND gate with two inputs (A, B) and one output (Y = A·B) loses one bit of information. This **loss of information leads to unavoidable energy dissipation**, which becomes critical in deep submicron and nano-scale technologies.[6][7]

2.1.2 Energy Dissipation in Irreversible Logic

According to **Landauer's Principle (1961)**, *every bit of information lost during computation leads to a minimum amount of heat generation*. When a logic operation is irreversible (i.e., the inputs cannot be reconstructed from the outputs), the lost information is converted into heat energy.[1][4]

Mathematically, Landauer stated that:

$$E_{min} = kT\ln(2)$$

where

- E_{min}= minimum energy dissipated per lost bit,
- k= Boltzmann's constant = $1.3806 \times 10^{-23}\ J/K$,
- T= absolute temperature (Kelvin).

At room temperature ($T = 300\ K$):

$$E_{min} \approx 2.8 \times 10^{-21}\ J \text{ per lost bit}$$

In large-scale circuits with billions of switching operations per second, this loss accumulates into **substantial power dissipation**, leading to:

- Increased junction temperature,
- Reduced reliability and lifespan,
- The need for active cooling and power management,
- Lower battery life in portable systems.

Thus, as transistor dimensions continue to scale down, the **thermal limitations of irreversible logic** have become a bottleneck for achieving higher speed and density in integrated circuits.

Conventional ALUs implement arithmetic operations using **irreversible full adders, subtractors, and multipliers**. Below are the mathematical formulations for each operation.[6][7][8]

2.1.3 Mathematical Representation of Conventional 8-bit ALU

(a) 8-bit Adder

An 8-bit binary adder adds two 8-bit binary numbers $A = (A_7, A_6, \ldots, A_0)$and $B = (B_7, B_6, \ldots, B_0)$, producing an 8-bit sum $S = (S_7, S_6, \ldots, S_0)$and a carry-out C_8.

For each bit:

$$S_i = A_i \oplus B_i \oplus C_i$$
$$C_{i+1} = (A_i \cdot B_i) + (B_i \cdot C_i) + (A_i \cdot C_i)$$

for $i = 0$ to 7, with initial carry $C_0 = 0$.

The overall **8-bit addition** can be written as:

$$S = A + B + C_{in}$$

Energy dissipation in a CMOS adder is dominated by **switching activity**, given by:

$$P_{dynamic} = \alpha C_L V_{DD}^2 f$$

where
α= switching factor,
C_L= load capacitance,
V_{DD}= supply voltage,
f= clock frequency.

(b) 8-bit Subtractor

The subtractor performs $A - B$, which is implemented as:

$$A - B = A + (\bar{B}) + 1$$

This uses the 2's complement method.

For each bit:

$$D_i = A_i \oplus B_i' \oplus B_{in}$$
$$B_{i+1} = (\bar{A}_\iota \cdot B_i') + (\bar{A}_\iota \cdot B_{in}) + (B_i' \cdot B_{in})$$

where $B_i' = \bar{B}_\iota$, and $B_{in}$is the borrow from the previous bit.

Just like the adder, every inversion and carry propagation involves transistor switching, leading to **dynamic power dissipation** proportional to V_{DD}^2.[6][7][8]

(c) 8-bit Multiplier

An 8-bit multiplier produces a 16-bit product $P = A \times B$. It is implemented using **partial product generation and accumulation**, which requires multiple AND gates and adders.

Each partial product:

$$P_{i,j} = A_i \cdot B_j$$

The product bits are then summed as:

$$P = \sum_{i=0}^{7} . \sum_{j=0}^{7} . (A_i \cdot B_j) \cdot 2^{i+j}$$

This architecture typically uses:

- 64 AND gates for partial products,
- 56 full adders and 8 half adders for accumulation.

Each gate transition in CMOS introduces **charging and discharging losses**, which contribute to high power consumption, especially at higher frequencies.

2.1.4 Energy Efficiency Analysis

In CMOS circuits, the total power dissipation $P_{total}$is expressed as:

$$P_{total} = P_{dynamic} + P_{static} + P_{short}$$
$$P_{dynamic} = \alpha C_L V_{DD}^2 f$$
$$P_{static} = I_{leak} \cdot V_{DD}$$
$$P_{short} = I_{short} \cdot V_{DD}$$

Even when switching activity is optimized, the **fundamental limit imposed by information loss** remains — an irreversible operation will always dissipate $kT\ln\ 2$joules per bit of lost information.

In contrast, **reversible ALUs**, constructed with gates such as **DKG**, **Peres**, and **BJN**, maintain input–output bijection, thereby eliminating the energy component associated with information loss. [6][7]

2.2 Evolution of Reversible Logic Gates

2.2.1 Historical Background

The evolution of **reversible logic** emerged from the intersection of **thermodynamics and computation**. The concept was first introduced in the early 1960s by **Rolf Landauer (1961)**, who established that *information loss in computation inevitably leads to energy dissipation*. His principle, now known as **Landauer's Principle**, quantifies the minimum amount of energy dissipated as heat when one bit of information is erased:

$$E_{min} = kT\ln\,(2)$$

where $k$is the Boltzmann constant and $T$is the absolute temperature in Kelvin.

This principle revealed that **information destruction has a physical cost**. For every bit of information lost, an amount of heat equal to $kT\ln\,(2)$joules is dissipated. This relationship bridged the gap between **information theory and thermodynamics**, suggesting that logically reversible operations could, in theory, be **thermodynamically lossless**.[1][4]

Building on Landauer's insight, **Charles H. Bennett (1973)** proposed that *if computation can be made logically reversible, it can be carried out without energy loss*. Bennett demonstrated that any irreversible algorithm can be simulated reversibly, provided that all intermediate information is preserved. This led to the foundation of **reversible computation**, where operations can be "uncomputed" to restore the initial state, ensuring that no information is destroyed.

In subsequent years, researchers such as **Edward Fredkin and Tommaso Toffoli (1980)** introduced the first practical reversible logic gates, laying the groundwork for **reversible digital circuits** and **quantum computing**. Since then, numerous reversible gate architectures have been developed to minimize power consumption, quantum cost, delay, and garbage outputs.

2.2.2 Concept of Reversibility

In an **irreversible logic gate**, the mapping between inputs and outputs is many-to-one — multiple input combinations may produce the same output. For instance, in an AND gate, both inputs (0, 1) and (1, 0) yield the same output (0); thus, the original inputs cannot be uniquely determined from the output.

In contrast, **reversible logic gates** establish a **one-to-one mapping** between input and output vectors:

$$f: \{0,1\}^n \rightarrow \{0,1\}^n$$

where $n$is the number of inputs/outputs.
Hence, the output uniquely determines the input, and no information is lost.

For a circuit to be reversible:

- The number of outputs must equal the number of inputs.
- Every input vector must produce a unique output vector.
- No fan-out (signal duplication) or feedback loops are allowed.

This **bijective property** ensures logical reversibility, enabling the design of **lossless computing circuits**.

2.3 Comparative Study of Existing Reversible ALUs

2.3.1 Overview

The Arithmetic Logic Unit (ALU) is the **core computational component** of any digital processor. Its design efficiency directly affects the overall **speed, power consumption, and area** of a computing system. While traditional ALUs based on irreversible CMOS logic incur significant **energy loss** due to information destruction (as per Landauer's principle), reversible ALUs have emerged as an efficient alternative by ensuring **information preservation** and **zero theoretical power dissipation**.

Over the past two decades, numerous researchers have proposed reversible ALU architectures that employ various reversible gates, including **Feynman**, **Toffoli**, **Fredkin**, **Peres**, **TR**, **DKG**, and **BJN**. These designs differ in terms of **quantum cost**, **garbage outputs**, **gate count**, and **speed**. This section presents a detailed **comparative analysis** of key reversible ALU architectures reported in the literature.[2][13][14][15][16]

2.3.2 Design Parameters for Comparison

Before comparing existing ALU designs, it is essential to establish the key evaluation metrics:

1. **Quantum Cost (QC):**
   The total number of primitive quantum operations (1×1 and 2×2 reversible gates) required to realize the circuit.

$$QC = \sum_{i=1}^{n} . QC_i$$

Lower quantum cost implies higher circuit efficiency.

2. **Garbage Outputs (GO):**
   Extra outputs required to preserve reversibility but not used for further computation.
   Reducing GOs minimizes circuit complexity and power overhead.

3. **Constant Inputs (CI):**
   Inputs fixed at logic '0' or '1' to achieve reversibility.
   Minimizing CI reduces unnecessary input complexity.

4. **Gate Count (GC):**
   Total number of reversible gates required to realize the ALU.
5. **Delay (D):**
   The maximum number of gate levels in the signal propagation path (critical path delay).
   It determines the circuit speed.
6. **Power Dissipation (P):**
   Ideally, near zero for reversible systems, but in practice, proportional to switching and leakage effects of the hardware realization.

2.3.3 Review of Early Reversible ALU Architectures

(a) Fredkin–Toffoli Based ALU (Early 2000s)

One of the first reversible ALU implementations employed **Fredkin** and **Toffoli gates** for arithmetic and logic operations.
Each arithmetic unit (adder/subtractor) was realized using cascaded Toffoli gates, while logical operations (AND, OR, XOR) were implemented using Fredkin gates.

- **Advantages:** Demonstrated the theoretical possibility of reversible computation for ALUs.
- **Limitations:** High gate count, excessive garbage outputs, and large quantum cost (≈70–90 for an 8-bit design).

This early design served as a **proof of concept** but was unsuitable for large-scale VLSI integration.

(b) Peres Gate-Based ALU (Deeptha et al., 2018)

To reduce quantum cost, **Peres gates** were introduced in ALU design. A Peres gate (3×3) combines XOR and AND functions efficiently, making it ideal for adders and multipliers.
In this design, arithmetic operations such as addition and subtraction were implemented using Peres-based full adders, while logic operations were constructed using Feynman and Fredkin gates.

Performance Summary:

- Gate count: 38
- Quantum cost: 64
- Garbage outputs: 18
- Delay: 12 gate levels

Observation:
The use of Peres gates reduced the quantum cost by approximately 30% compared to the Toffoli-based approach; however, garbage outputs remained relatively high due to signal duplication constraints.

(c) DKG-Based ALU (Lenin Gopal et al., 2021)

The **D.K. Gate (DKG)** was introduced to address the issue of excessive garbage outputs in Peres-based circuits. The DKG gate is a 4×4 reversible gate capable of realizing both sum and carry within a single gate.

This ALU design is used:

- DKG gates for arithmetic operations (adders/subtractors),
- Fredkin gates for control logic,
- Feynman gates for signal copying.

Performance Summary:

- Gate count: 32
- Quantum cost: 52
- Garbage outputs: 12
- Constant inputs: 8
- Delay: 9 gate levels

Observation:
This approach achieved **better scalability** and **lower garbage generation**, making it suitable for **8-bit and 16-bit ALU designs**. However, the delay could still be optimized through advanced pipelining or parallelism.

(d) TR Gate and BJN Gate-Based ALU (Thapliyal et al., 2022)

Later designs combined **TR gates** and **BJN gates** for improved arithmetic and comparison operations.
The TR gate efficiently implements XOR–AND combinations, while the BJN gate performs both sum and carry functions simultaneously.

This hybrid design achieved a significant reduction in **quantum cost** and **delay**, making it one of the most optimized reversible ALU architectures.

Performance Summary:

- Gate count: 28

- Quantum cost: 44
- Garbage outputs: 10
- Constant inputs: 6
- Delay: 7 gate levels

Observation:
The TR–BJN combination proved effective for **low-power reversible comparators and ALUs**, offering the best trade-off among quantum cost, garbage outputs, and speed.

(e) Programmable Reversible ALU (Sameer Suman et al., 2023)
A more recent design introduced a **programmable reversible ALU** using **Fredkin, Peres, and TR gates**. The control logic was also implemented reversibly, allowing dynamic selection between arithmetic and logical operations.

Performance Summary:

- Gate count: 40
- Quantum cost: 60
- Garbage outputs: 14
- Constant inputs: 8
- Delay: 10 gate levels

Observation:
While programmable, the added control circuitry slightly increased the gate count and quantum cost. Nonetheless, it provided greater flexibility for integration into **reversible processors**.

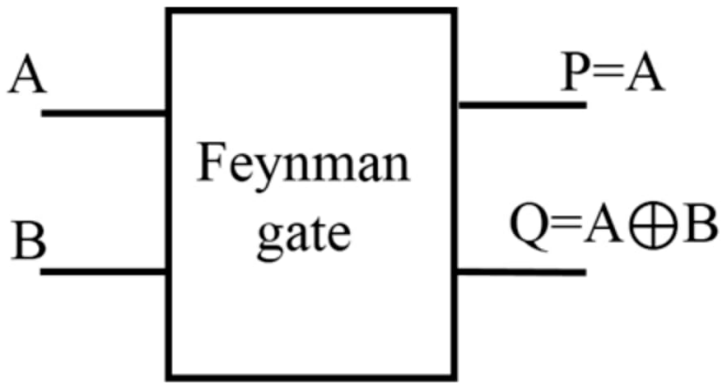
2.4 Basic Reversible Logic Gates (Feynman, Fredkin, Toffoli, Peres, etc.)

2.4.1 The Feynman Gate (Controlled-NOT or CNOT Gate)

The **Feynman Gate**, introduced by Richard Feynman, is a 2×2 reversible gate with inputs (A, B) and outputs (P, Q):

$$P = A$$
$$Q = A \oplus B$$

It acts as a **signal copier** and **inverter** depending on the value of A.



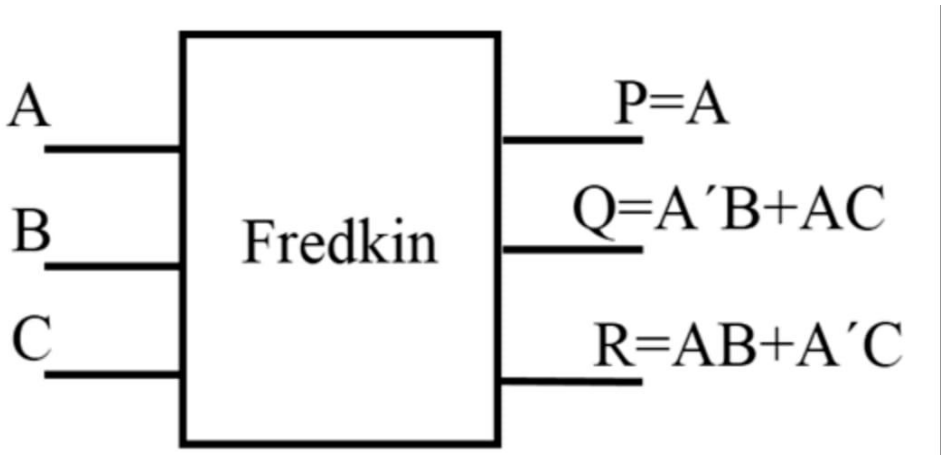
Since fan-out is not allowed in reversible circuits, the Feynman gate is essential for duplicating signals. Its **quantum cost** is 1, making it the simplest and most fundamental reversible gate.

2.4.2 The Fredkin Gate

Proposed by **Edward Fredkin and Tommaso Toffoli**, the **Fredkin gate** is a 3×3 reversible gate that performs a **controlled swap operation**.

For inputs (A, B, C), the outputs are:

$$P = A$$
$$Q = A'B + AC$$
$$R = A'C + AB$$



When A = 0, the gate passes B and C unchanged; when A = 1, it swaps B and C. This makes it highly suitable for **multiplexing, demultiplexing, and data routing** applications in reversible designs.

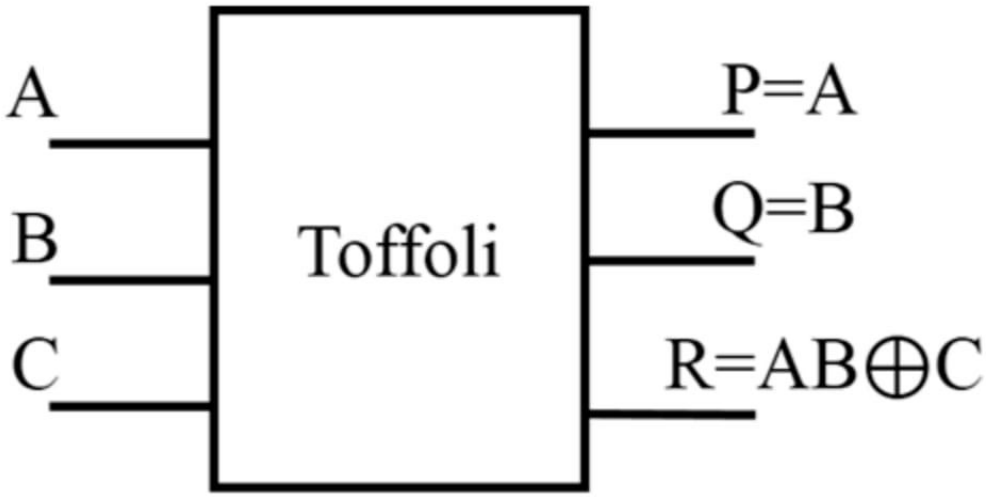
The Fredkin gate marked a major milestone because it showed that **universal computation could be achieved with conservative logic** — preserving both information and bit count.

2.4.3 The Toffoli Gate

The **Toffoli Gate** (also known as the **Controlled-Controlled-NOT** or **CCNOT gate**) is another foundational 3×3 reversible gate. It performs a NOT operation on the third input if and only if both the first and second inputs are 1:

$$P = A$$
$$Q = B$$
$$R = C \oplus (A \cdot B)$$

It is **universal for classical reversible computation**, meaning any reversible Boolean function can be realized using only Toffoli gates.



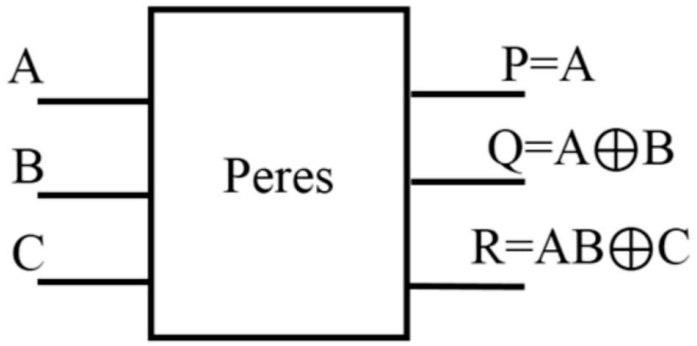
The Toffoli gate forms the **basis for quantum logic**, where it can be represented as a unitary transformation in the quantum domain. Its quantum cost is 5 when implemented using basic 2×2 quantum primitives.

2.4.4 The Peres Gate

Introduced by **Asher Peres (1985)**, the **Peres Gate** combines the functionality of Feynman and Toffoli gates, achieving **lower quantum cost** and **fewer garbage outputs**.

For inputs (A, B, C):

$$P = A$$
$$Q = A \oplus B$$
$$R = (A \cdot B) \oplus C$$



The Peres gate has a **quantum cost of 4**, compared to 5 for the Toffoli gate, making it a preferred choice for **low-power arithmetic circuits** such as adders, subtractors, and multipliers. It is often referred to as the *building block of efficient reversible arithmetic units*.

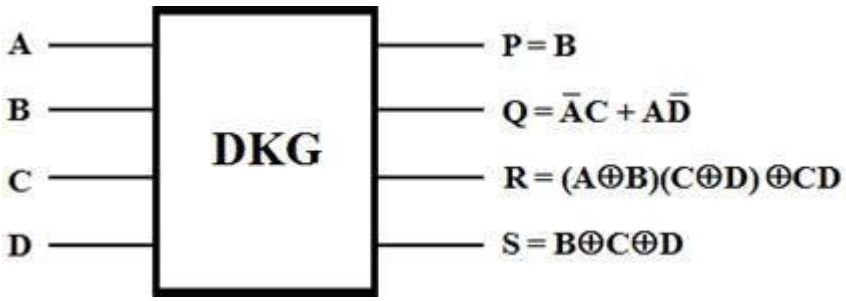
2.5 Advanced Reversible Gates (DKG, TR, BJN, PPG)

Over time, several **improved reversible gates** were proposed to address limitations such as high garbage outputs and quantum costs. The goal of these gates is to realize complex Boolean functions with minimal resources.

2.5.1 DKG Gate (D. K. Gate)

The **DKG gate** is a 4×4 reversible gate that extends the concept of the Peres gate to perform multiple reversible operations efficiently.
It is defined as:

$$P = B$$
$$Q = \bar{A}.\mathrm{C} + \mathrm{A}\bar{D}$$
$$R = (A \oplus B).(C \oplus D) \oplus (C.D)$$
$$S = (B \oplus C \oplus D)$$

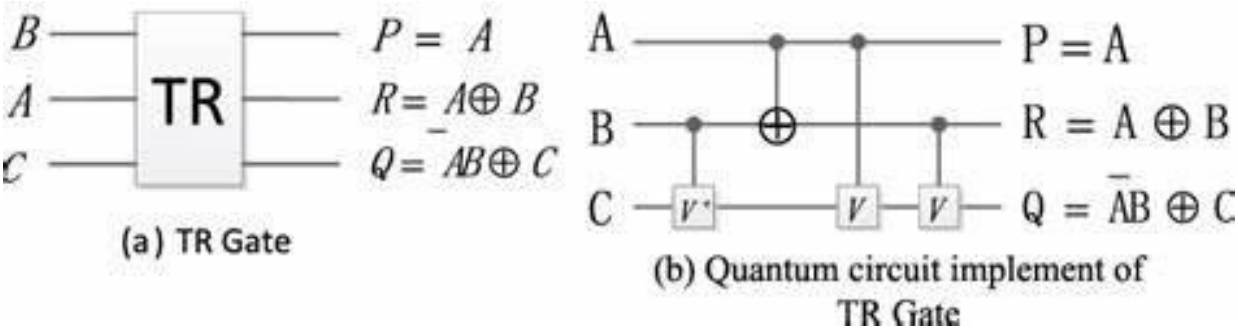


The DKG gate enables **compact adder and subtractor designs** by generating both sum and carry outputs within a single reversible structure. It offers reduced garbage outputs and quantum cost, making it a powerful gate for **multi-bit arithmetic circuits**.

2.5.2 TR Gate (Thapliyal–Ranganathan Gate)

Proposed by **Thapliyal and Ranganathan**, the **TR Gate** is a 3×3 reversible gate optimized for arithmetic logic operations:

$$P = A$$
$$Q = A \oplus B$$
$$R = (A \cdot B) \oplus C$$



(a) TR Gate

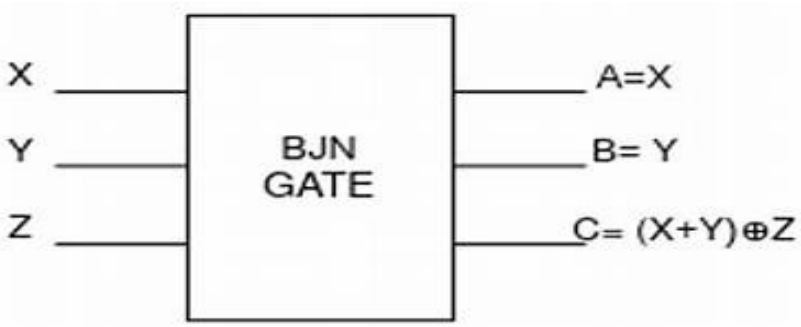
(b) Quantum circuit implement of TR Gate

It is primarily used in **reversible full adders** and **ALUs**, achieving better trade-offs between delay and quantum cost. It plays a key role in the design of **reversible comparators** and **control logic**.

2.5.3 BJN Gate

The **BJN Gate** is a 4×4 reversible gate designed for low-power and high-efficiency arithmetic computation. For inputs (A, B, C, D):

$$P = A$$
$$Q = B$$
$$R = (A + B) \oplus C$$

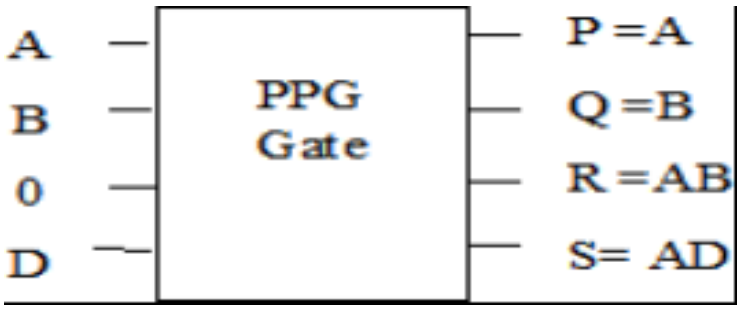


The third output represents the **sum,** and the fourth output represents the **carry** of a reversible full adder. It is used in **reversible comparator and ALU designs** to optimize speed and minimize garbage bits.

2.5.4 PPG Gate (Partial Product Generator Gate)

The **PPG Gate** is a 4×4 reversible gate, designed specifically for use in **reversible multiplier circuits**.
For inputs (A, B, C, D):

$$P = A$$
$$Q = B$$
$$R = A \cdot B$$
$$S = (A \cdot B) \oplus C$$



This gate efficiently generates **partial products** with minimal garbage outputs, which are later summed using reversible adders to produce the final product. Its quantum cost and delay are optimized for **VLSI and quantum multiplier implementations**.

2.6 Key Design Parameters

The performance of any **reversible logic circuit**, particularly in the design of **Arithmetic Logic Units (ALUs)**, is evaluated based on a set of critical design parameters that determine the **efficiency, scalability, and energy performance** of the architecture. These parameters directly influence the physical realizability of the circuit in **low-power VLSI** and **quantum computing** applications.

Quantum Cost (QC)

Quantum Cost represents the **number of basic quantum operations** (1×1 or 2×2 reversible primitives) required to implement a reversible circuit. Each reversible gate has an associated QC value — for example, the Feynman gate has a QC of 1, the Peres gate 4, and the Toffoli gate 5.

A lower quantum cost corresponds to **higher hardware efficiency** and **reduced energy consumption**.

$$QC_{total} = \sum_{i=1}^{n} QC_i$$

Minimizing QC is a primary goal in optimizing reversible ALU designs.

Garbage Outputs (GO)

In reversible logic, all outputs must be unique to maintain a one-to-one input–output mapping. However, some intermediate outputs are not used in further computation; these are referred to as **garbage outputs**. Excessive garbage increases circuit complexity and routing overhead. Hence, an optimized design strives to **minimize the number of garbage outputs (GOs)** while maintaining reversibility.

$$GO = N_{out(total)} - N_{out(useful)}$$

Constant Inputs (CI)

Constant Inputs (also called **ancilla bits**) are inputs fixed to logic '0' or '1' to ensure reversibility or generate required control signals. A higher CI count leads to **larger input overhead** and **increased static complexity**. Therefore, minimizing CI is essential for compact and energy-efficient circuit realization.

Delay (D)

Delay measures the **maximum number of sequential gate levels** between input and output, representing the circuit's critical path. In reversible ALUs, delay is minimized by cascading low-depth gates like **DKG** and **TR** to achieve **faster computation** with minimal gate dependencies.

$$D_{total} = \max \text{ (path length between input and output)}$$

Hardware Complexity (HC)

Hardware complexity refers to the total number of basic logical operations (AND, XOR, NOT) required to realize the circuit. It is an indirect indicator of both **area**

and **power utilization**. Lower hardware complexity contributes to smaller circuit size and reduced switching power.

Power Dissipation (P)

Although reversible circuits theoretically dissipate zero energy due to information preservation, in practical CMOS or FPGA realizations, **dynamic and leakage power** still exist. Total power is estimated as:

$$P_{total} = \alpha C_L V_{DD}^2 f + I_{leak} V_{DD}$$

Reversible architectures inherently reduce $P_{total}$by minimizing redundant switching and information loss.

CHAPTER – 3 Methodology and Design

3.1 Design Flow Overview

The overall design process followed a **bottom-up hierarchical approach**, starting from transistor-level design and progressing to the complete reversible ALU architecture. Initially, **transmission gates (Fig. 1)** were designed using **Pass Transistor Logic (PTL)**, where a parallel combination of NMOS and PMOS devices was used to achieve full voltage swing and bidirectional switching capability. The control signal **S** and its complement $\overline{\mathbf{S}}$, generated by a CMOS inverter, were applied to the transistor gates to form an efficient low-power switch.

A transmission gate, also known as a pass gate, is formed by connecting an NMOS and a PMOS transistor in parallel to allow both strong logic '0' and logic '1' levels to pass efficiently. The NMOS transistor conducts well when passing a low voltage (logic 0), while the PMOS transistor conducts well when passing a high voltage (logic 1). By combining them, the transmission gate can pass signals of both logic levels without significant degradation. The gate control signals for the NMOS and PMOS are complementary — if the NMOS gate is driven by signal **g**, the PMOS gate is driven by its complement $\overline{\mathbf{g}}$. This complementary control ensures that when the gate is enabled, both transistors conduct simultaneously, allowing the input signal to pass through; when disabled, both transistors are off, isolating the input from the output. This concept forms the basis of **Complementary Pass-Transistor Logic (CPL)** or **dual-line pass-transistor logic**, where each signal is represented by both its true and complement forms, enabling high-speed, low-power digital designs.

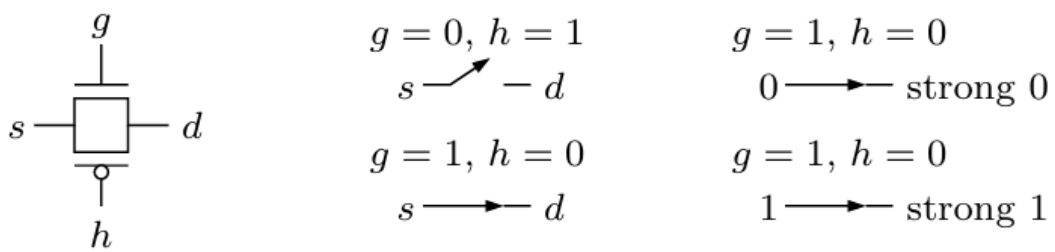


Fig. 1 Transmission Gate Diagram

Adiabatic switching is a technique used to reduce power dissipation in transistor circuits by reusing the energy stored in signals during switching. The term *adiabatic*, borrowed from physics, refers to a process where no energy is lost as

heat; in electronics, it implies that the energy dissipated during transistor switching approaches zero as the switching time increases indefinitely. This is achieved by ensuring that the control signal (**g**) changes gradually rather than abruptly, allowing a smooth voltage transition from logic 0 to logic 1 (or vice versa). For adiabatic operation, two key rules must be followed:

1. A transistor should never be turned on when there is a significant voltage difference between its source and drain, and
2. It should never be turned off while a substantial current is flowing through it. These conditions minimize energy loss during the charge transfer process. However, it's essential to note that while adiabatic circuits aim for low power dissipation, they are not inherently reversible; conversely, reversible circuits, which conserve information, are not always adiabatic.

Below are the gates implemented using this **CPL** and adiabatic switching

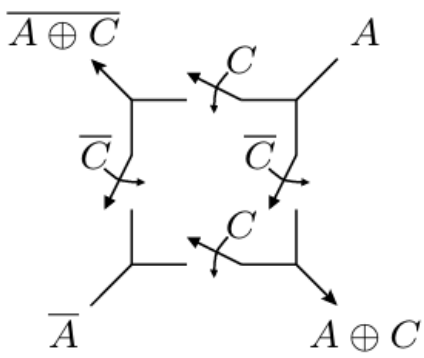


Fig.2 Implementation of the Feynman gate using 4-pass gates

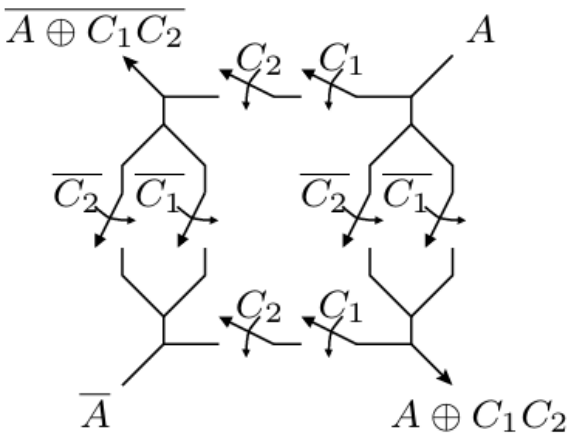


Fig.3 Implementation of the Toffoli using 8-pass gates

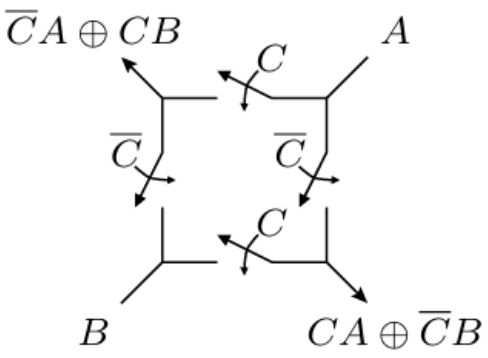


Fig. 4 Implementation of the Fredkin gate in R-CPL using 8-pass gates

The Feynman, Toffoli, and Fredkin gates form the fundamental building blocks of reversible logic design. The **Feynman gate** (also known as the Controlled-NOT gate) provides signal copying and inversion, which are essential for fan-out and logic complementation in reversible circuits. The **Toffoli gate**, being universal for reversible computation, can implement all basic Boolean functions such as AND, OR, and NOT in a reversible manner. The **Fredkin gate**, on the other hand, performs conditional swapping and is efficient for conservative logic designs where the number of logic 1s remains constant. Together, these three gates provide the core functionality required to construct any complex reversible logic circuit. All other reversible gates, such as **the Peres, New, and T gates, are essentially derived or optimized combinations of these fundamental ones, tailored for specific design trade-offs, including** reduced quantum cost, lower delay, or minimal garbage outputs.

3.2 Schematics of the gates on Cadence

All the reversible logic gates — **Feynman**, **Toffoli**, **Fredkin**, and **DKG** — were designed and simulated at the transistor level using **Cadence Virtuoso**. Each gate was implemented using **transmission gates (CMOS pass-transistor logic)** to ensure full logic swing, low power dissipation, and minimal delay.

3.2.1 The Feynman Gate

The Feynman gate (Fig. 5), also known as the Controlled-NOT (CNOT) gate, was implemented in Cadence using CMOS transmission logic. It features parallel-connected NMOS and PMOS transistors that together form a transmission gate structure, enabling both strong logic '0' and logic '1' to pass efficiently. The gate has two inputs (A and B) and two outputs (P = A, Q = A ⊕ B). In the schematic, the XOR function is implemented using pass transistors controlled by input A, which enables the conditional inversion of B. This design provides full logic swing, prevents signal degradation, and minimizes power dissipation, which are crucial for reversible operations.

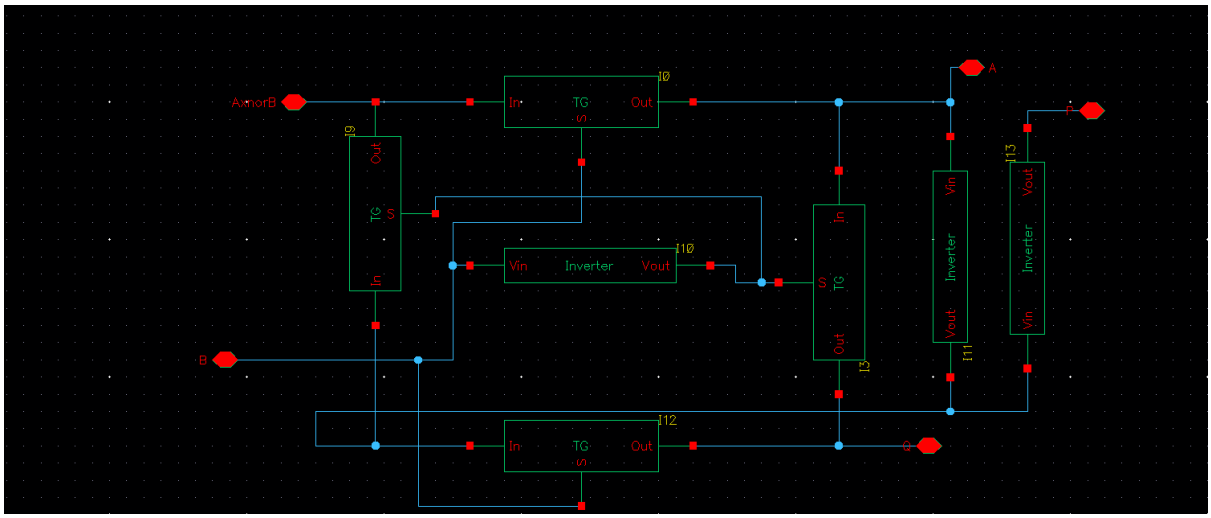


Fig. 5 The Feynman Gate

3.2.2 Toffoli Gate:

The Toffoli gate (Fig. 6), also known as the Controlled-Controlled-NOT (CCNOT) gate, was designed using cascaded transmission gates to achieve the required three-input, three-output mapping. The schematic implements the function (P = A, Q = B, R = AB ⊕ C), where the output bit C is inverted only when both A and B are high. The control logic was designed using two layers of transmission gate networks to selectively propagate signals, ensuring reversibility and correct logic functionality. The transistor-level design preserves the charge and logic state across switching events, reducing energy

dissipation and confirming the gate's suitability for low-power, reversible computing.

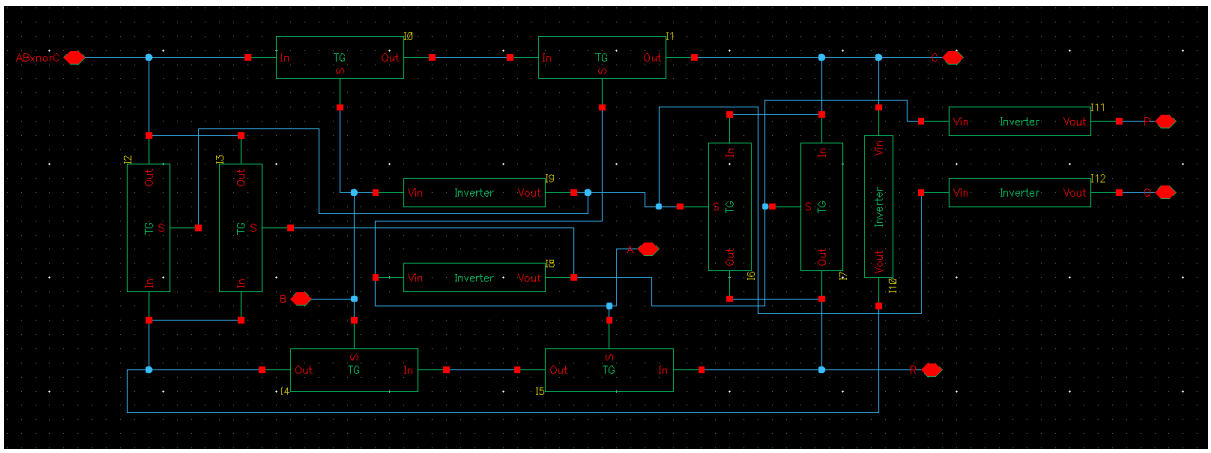


Fig. 6 Toffoli Gate

3.2.3 Fredkin Gate:

The Fredkin gate (Fig. 7) schematic was developed using CMOS-based pass-transistor logic to realize a conditional swap function. It has one control input and two data inputs, with outputs defined as (P = A, Q = A′B + AC, R = A′C + AB), meaning the two data inputs are swapped when the control signal A is high. The transistor network in Cadence was arranged such that signal paths dynamically change according to the control voltage, ensuring faithful signal transfer with minimal leakage. The gate conserves the number of logical '1's, making it both reversible and conservative. Simulation results validated correct signal switching and full voltage restoration at the outputs.

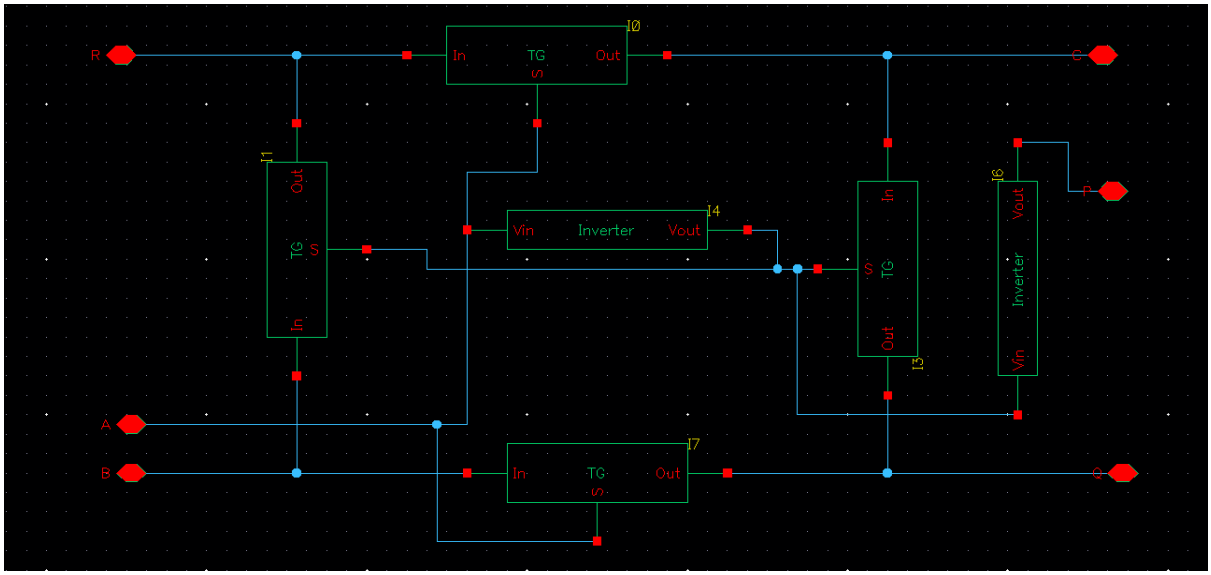


Fig. 7 Fredkin Gate

3.2.4 Peres Gate:

The Peres gate **(Fig.8)** is one of the most efficient and widely used reversible logic gates due to its low quantum cost and ability to perform both XOR and AND operations within a compact structure. It is a **3×3 reversible gate** with inputs (A, B, C) and outputs defined as ($P = A$, $Q = A \oplus B$, $R = AB \oplus C$). Functionally, the Peres gate combines the actions of a **Feynman gate** (for XOR operation) and a **Toffoli gate** (for controlled-AND operation) into a single unit, thereby reducing hardware complexity and eliminating unnecessary outputs. In Cadence, the Peres gate schematic was implemented using transmission gate logic, comprising complementary NMOS and PMOS transistors, to ensure full voltage swing and minimal signal degradation. The gradual switching of control signals enables energy-efficient transitions while maintaining logical reversibility. Simulation results confirmed accurate implementation of all output functions, demonstrating the gate's suitability for **low-power reversible arithmetic circuits**, such as adders, subtractors, and ALUs, where it provides an optimal trade-off between performance, area, and power consumption.

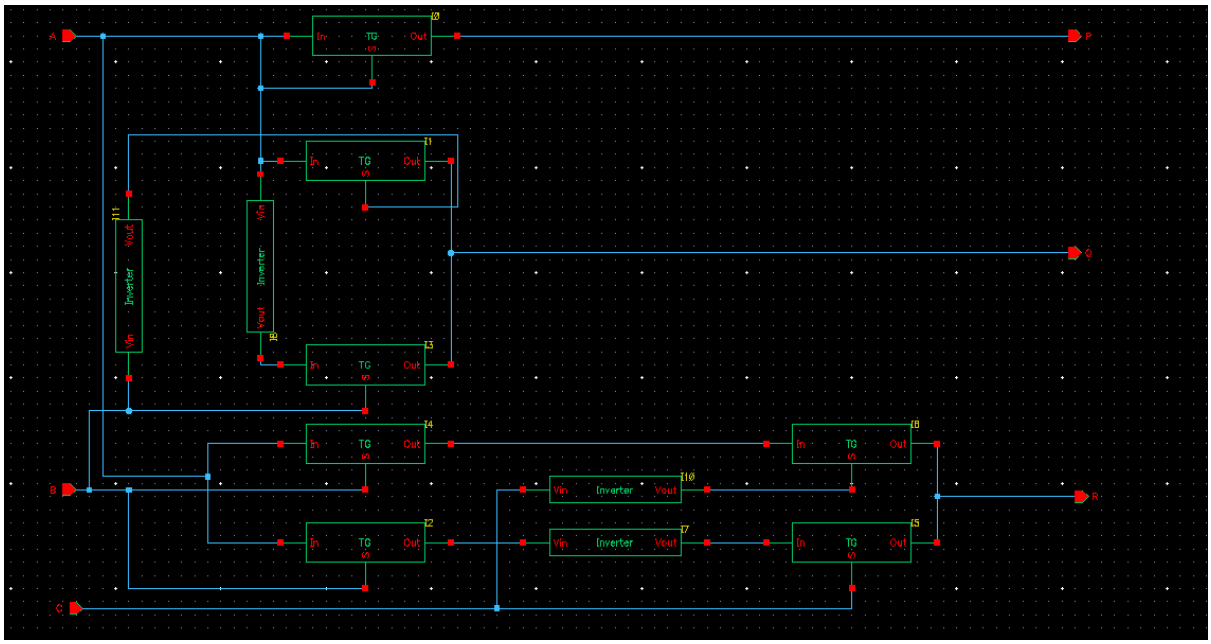


Fig. 8 Peres Gate

3.2.5 DKG Gate:

The DKG (Double Kunh Gate) (Fig. 9) schematic integrates the essential functionalities of the Feynman, Toffoli, and Fredkin gates into a single optimized design. It was implemented using a compact CMOS transmission structure that minimizes garbage outputs, constant inputs, and overall transistor count. The gate performs both XOR and AND-controlled operations, making it

ideal for constructing reversible full adders. In Cadence, the DKG schematic was verified for correct logic transitions through transient and DC analyses. The results demonstrated robust output levels, symmetric rise and fall times, and low energy dissipation, confirming the DKG gate's efficiency and reliability as a building block for larger reversible circuits.

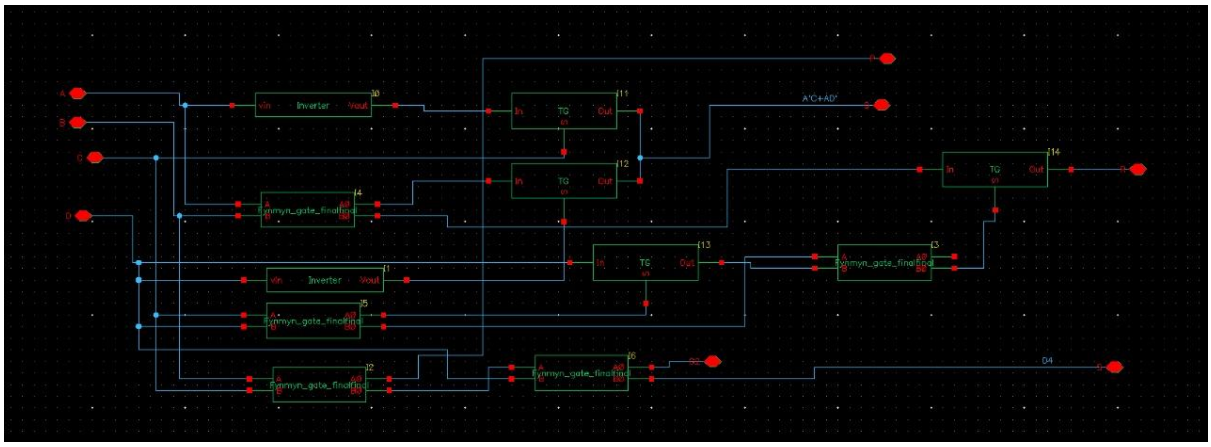


Fig. 9 DKG Gate

3.2.6 TR Gate:

The **TR gate** (Fig. 10) (Thapliyal–Ranganathan gate) is a **3×3 reversible logic gate** designed to perform complex logical operations with reduced hardware complexity and minimal power dissipation. Its input vector is (A, B, C) and the corresponding output vector is (P = A, Q = A ⊕ B, R = (A·B′) ⊕ C). This configuration allows the TR gate to simultaneously perform XOR and AND-controlled operations, making it highly efficient for constructing reversible full adders, subtractors, and multiplexers. In **Cadence**, the TR gate schematic was implemented using **transmission gate-based CMOS logic**, where NMOS and PMOS transistors operate in complementary fashion to pass both logic '0' and logic '1' levels without degradation.

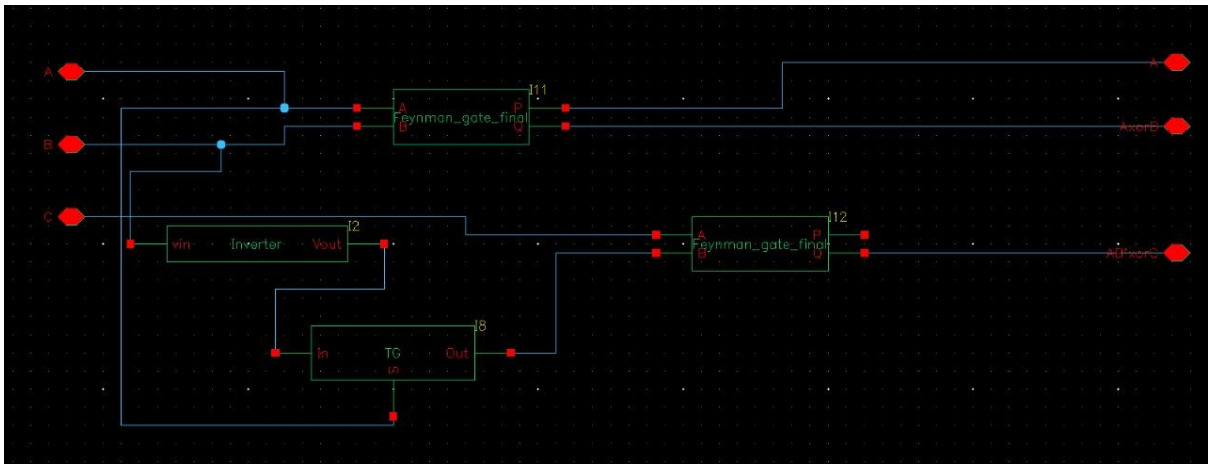


Fig. 10 TR Gate

3.2.7 BJN Gate:

The **BJN gate** (Fig. 11) (proposed by Bruce, Jha, and Nayeem) is a **4×4 reversible logic gate** designed to optimize reversible arithmetic operations by reducing **garbage outputs**, **constant inputs**, and **quantum cost**. It accepts four inputs (A, B, C, D) and produces four outputs typically defined as ($P = A$, $Q = A \oplus B$, $R = (A \cdot B) \oplus C$, $S = ((A \oplus B) \cdot C) \oplus D$). This combination of operations enables the BJN gate to perform complex Boolean functions such as **full addition, subtraction, and controlled logic** within a single reversible unit. Structurally, it integrates the functional behavior of **Feynman**, **Toffoli**, and **Peres** gates, allowing it to achieve higher computational capability while maintaining reversibility.

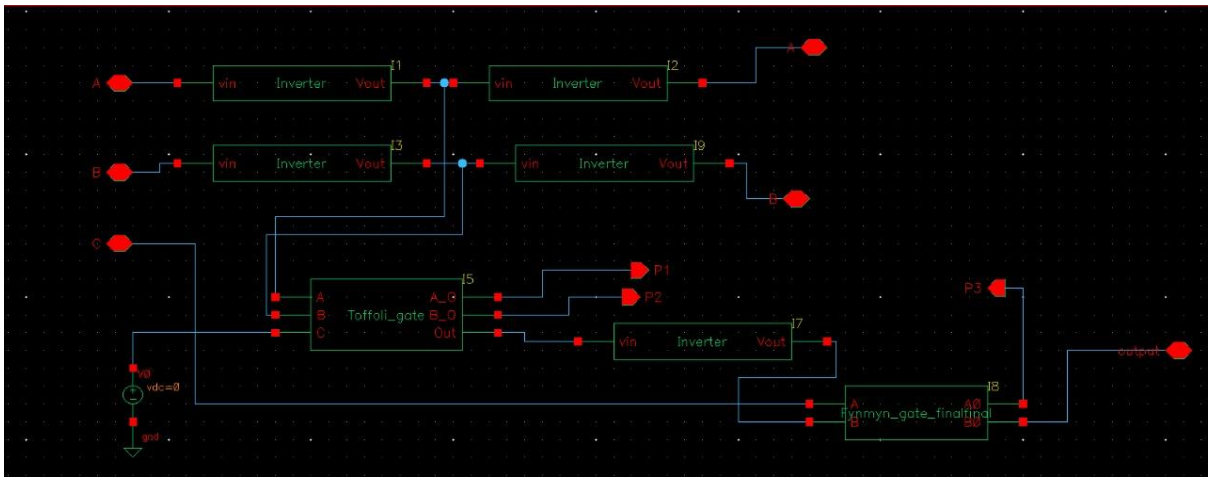


Fig. 11 BJN Gate

Each reversible quantum gate (Feynman, Toffoli, Fredkin, Peres, TR, and BJN) was realized using combinations of these transmission gates and inverters. These gates form the **functional backbone** of the proposed reversible ALU. The simulation and verification of each gate were performed at the transistor level using **Cadence Virtuoso**, ensuring correct logical operation and maintaining reversibility (i.e., a 1:1 input-output mapping).

3.3 Design of 1-bit and 8-bit Reversible Adders (DKG-based)

The **1-bit DKG-based reversible full adder** was designed in **Cadence Virtuoso** using the **DKG (Double Kunh Gate)** as the core computational unit. The DKG gate efficiently produces both the **Sum** and **Carry** outputs while maintaining reversibility and minimizing energy loss. As shown in Fig. 12(a), the schematic consists of input terminals A, B, and C, which are connected to the DKG block, with an ancilla bit 0 for addition. The output lines are further processed through inverters to ensure full logic swing and signal restoration before generating the final **Sum** and **Carry** outputs. The circuit operates with complementary pass-transistor logic to maintain strong logic levels and reduced power dissipation.

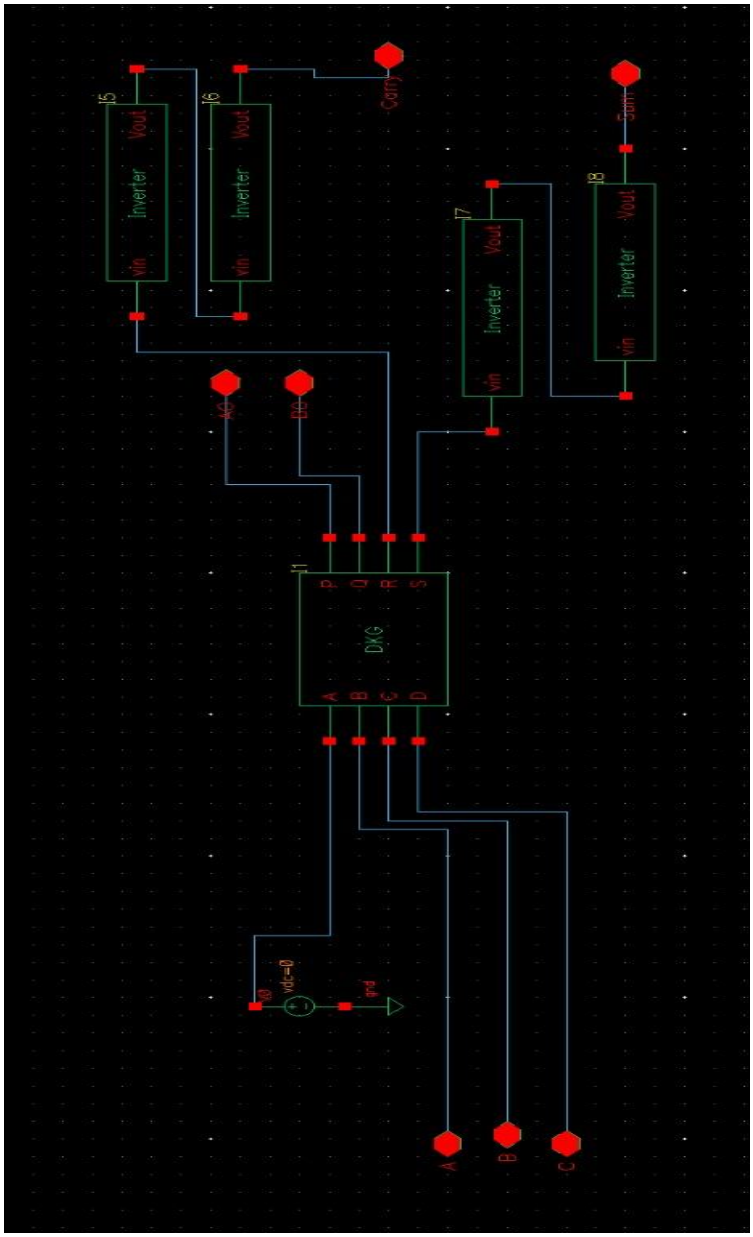


Fig. 12(a) 1-bit DKG adder

$$P = B$$
$$Q = \bar{A}.\mathrm{C} + \mathrm{A}\bar{D}$$
$$R = (A \oplus B).(C \oplus D) \oplus (C.D)$$
$$S = (B \oplus C \oplus D)$$

1. **Substitute $A = 0$ into each expression.**
   Replace every occurrence of A with 0 and $\bar{A}$ with 1.
2. **Simplify P.**
   $P = B$— unchanged because P does not depend on A.
3. **Simplify Q.**
   $$Q = \bar{A}.C + A.\bar{D} = 1.C + 0.\bar{D} = C$$
4. Simplify the XOR term in R.
   $$A \oplus B = 0 \oplus B = B$$
   So substitute into R:
   $$R = B(C \oplus D) \oplus CD$$
5. Simplify S.
   $$S = B \oplus C \oplus D - \ unchanged\ by\ A.$$
6. Final equation

$$\boxed{P = B, Q = C, R = B(C \oplus D) \oplus CD, S = B \oplus C \oplus D}$$

The **8-bit reversible adder**, shown in Fig. 12(b), is constructed by cascading eight 1-bit DKG-based adder modules in a **ripple-carry configuration**. Each 1-bit block generates its Sum output and passes the Carry to the next stage, ensuring bit-wise addition in a fully reversible manner. This hierarchical design approach simplifies layout implementation and ensures scalability. The circuit utilizes identical DKG-based cells for all eight bits, achieving a uniform structure with reduced garbage outputs and efficient transistor utilization. Transient analysis verified the correct addition operation across all bit positions, confirming accurate Sum and Carry propagation with minimal delay and low power consumption.

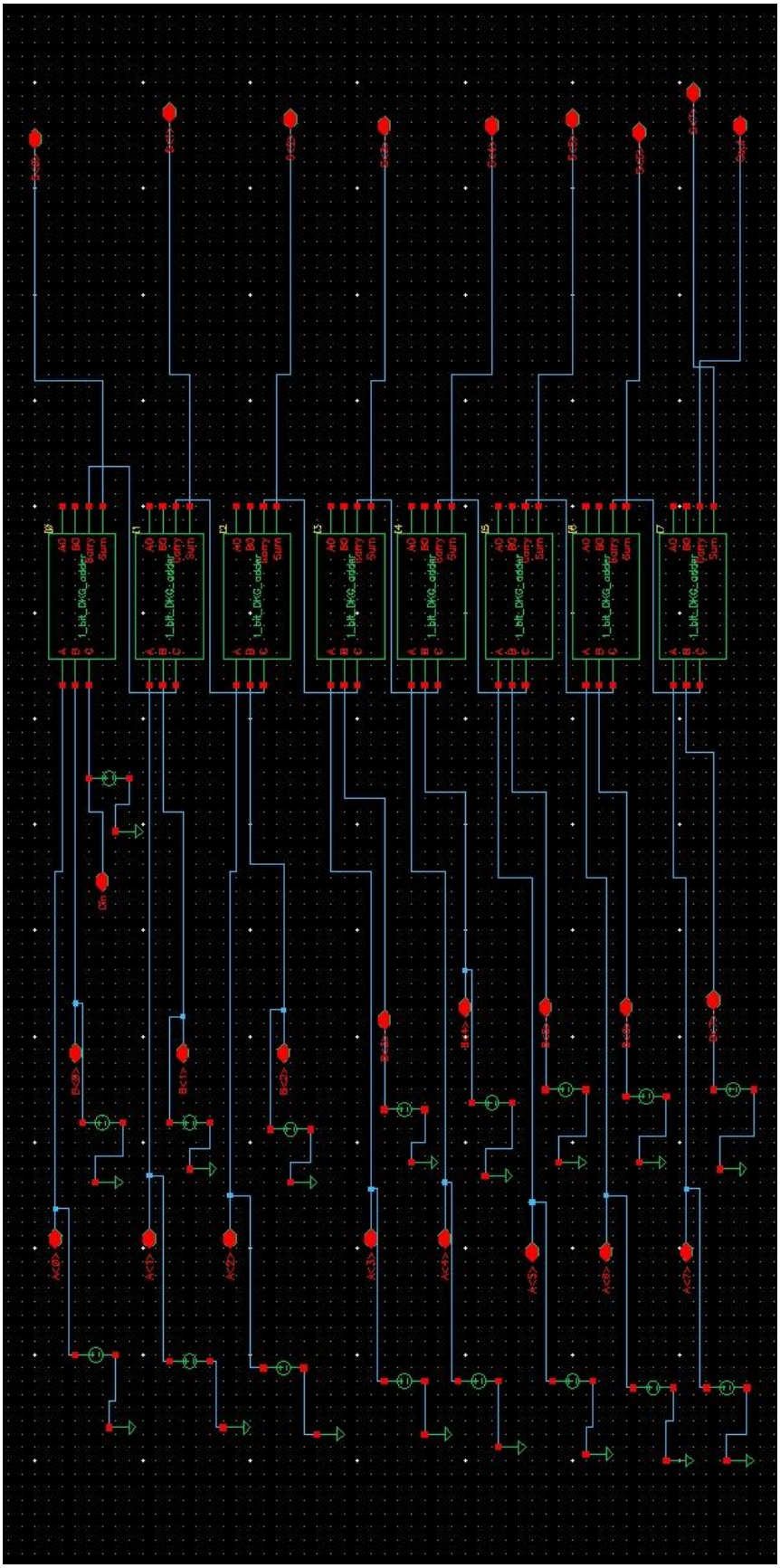


Fig. 12(b) 8-bit adder cascaded using 1-bit DKG adder

3.4 Design of 1-bit and 8-bit Subtractors

The **1-bit DKG-based reversible full subtractor** (Fig. 13(a)) was designed using the same DKG gate structure employed in the adder circuit, but with the control input **A fixed to logic '1' (ancilla = 1)** to enable subtraction functionality. The DKG gate equations $P = B$, $Q = \bar{A}C + A\bar{D}$, $R = (A \oplus B)(C \oplus D) \oplus CD$, and $S = B \oplus C \oplus D$were modified for $A = 1$, resulting in $P = B$, $Q = \bar{D}$, $R = \bar{B}(C \oplus D) \oplus CD$, and $S = B \oplus C \oplus D$. Here, S represents the **Difference** output, while R corresponds to the **Borrow-out**. The Boolean expression for Raligns with the conventional full-subtractor logic:

$$\text{Borrow}_{out} = \bar{B}C + (C \oplus B)'D = \bar{B}(C \oplus D) \oplus CD,$$

which confirms that the DKG gate performs accurate subtraction while maintaining reversibility. The use of the ancilla input $A = 1$ ensures proper control flow and energy-efficient operation, while $P$and Q act as reversible auxiliary outputs (garbage), preserving information for backward traceability.

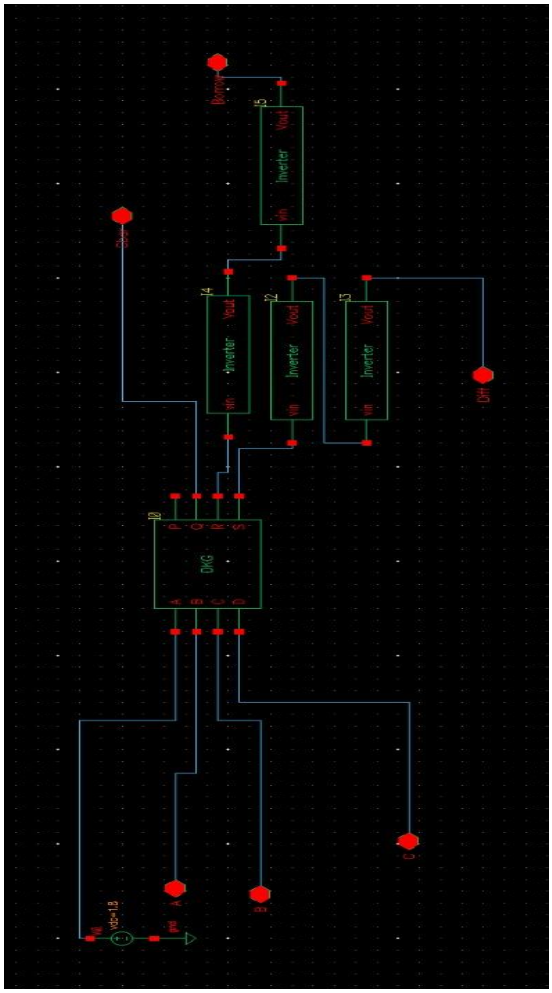


Fig. 13(a) 1-bit Subtractor using DKG gate

1. **State the original DKG output equations**.

$$\begin{aligned} P &= B \\ Q &= \bar{A}\,C + A\,\bar{D} \\ R &= (A \oplus B)\,(C \oplus D) \oplus (C\,D) \\ S &= B \oplus C \oplus D \end{aligned}$$

2. **Substitute** $A = 1$ into each expression. Replace $\bar{A}$ with 0 and A with 1.
3. **Simplify** P**.**
   $P = B$— unchanged because Pdoes not depend on A.
4. **Simplify** Q**.**

$$Q = \bar{A}\,C + A\,\bar{D} = 0 \cdot C + 1 \cdot \bar{D} = \bar{D}.$$

5. **Simplify the XOR term in** R**.**

$$A \oplus B = 1 \oplus B = \bar{B}.$$

6. **Substitute into** $\boldsymbol{R}$:

$$R = \bar{B}\,(C \oplus D) \oplus CD.$$

7. **Simplify** S**.**
   $S = B \oplus C \oplus D$— unchanged by A.
8. **Summarize final equations (for** $A = 1$**).**

$$\boxed{P = B, Q = \bar{D}, R = \bar{B}(C \oplus D) \oplus CD, S = B \oplus C \oplus D}$$

The **8-bit reversible subtractor** (Fig. 13(b)) was constructed by cascading eight such 1-bit DKG subtractor modules in a **ripple-borrow configuration**, similar to the ripple-carry structure in adders. Each stage receives its own inputs B_i(minuend), C_i(subtrahend), and borrow-in D_i, and produces a difference output $S_i$and borrow-out R_i, which is passed to the next higher bit. The most significant stage provides the final borrow output. All stages utilize the same constant input cap, An input $A = 1$, ensuring uniform reversible operation throughout the 8-bit structure. This modular design reduces hardware complexity and garbage outputs while maintaining low power and delay. Simulation results validated the correct bitwise subtraction, confirming that the DKG-based 8-bit reversible subtractor is efficient, fully reversible, and suitable for integration in a reversible ALU.

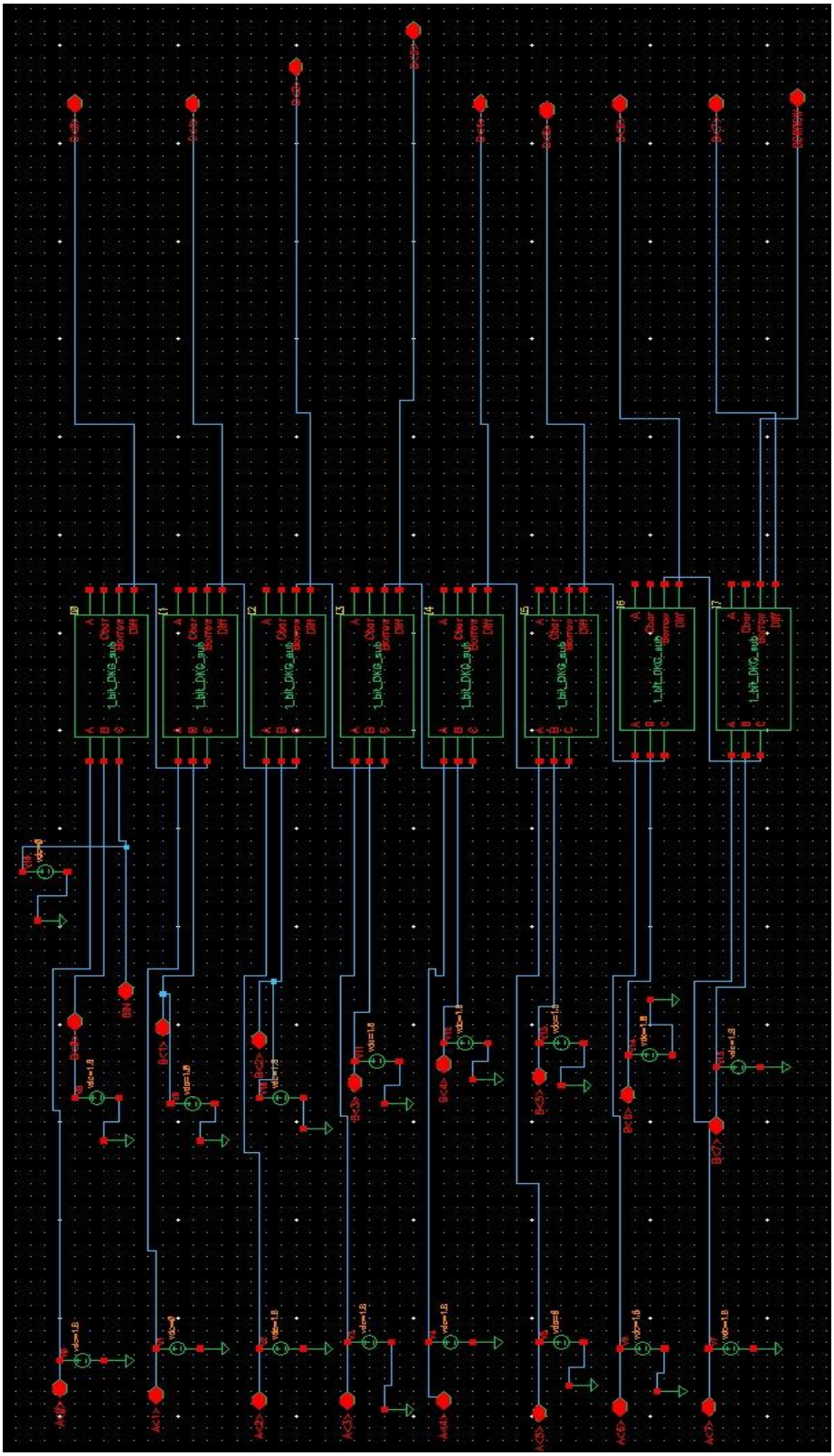


Fig. 13(b) 8-bit subtractor cascaded using 1-bit Subtractor

3.5 Design of 2-Bit Reversible Multiplier (Peres Gate-based)

A **multiplier** is a fundamental arithmetic circuit widely used in processors, digital signal processing, and cryptographic systems. It performs repeated addition to compute the product of two binary numbers. In a conventional irreversible circuit, multiplication leads to significant power dissipation because information is lost during logic transitions. However, in **reversible logic**, every input vector uniquely maps to an output vector, eliminating information loss and allowing for energy recovery in accordance with **Landauer's principle**. Hence, implementing a 2-bit multiplier using reversible gates ensures low power consumption and energy-efficient computation.

The operation of a 2-bit binary multiplier involves generating **partial products** and then adding them in binary form. Let the two inputs be $A = A_1A_0$and $B = B_1B_0$. The binary multiplication can be expressed as:

$$A_1A_0 \times B_1B_0 = (A_1 \times B_1)2^2 + (A_1 \times B_0 + A_0 \times B_1)2^1 + (A_0 \times B_0)2^0.$$

The corresponding Boolean expressions for each product bit are:

$$\begin{aligned} P_0 &= A_0B_0, \\ P_1 &= (A_1B_0) \oplus (A_0B_1), \\ P_2 &= (A_1B_1) \oplus (A_1B_0 \cdot A_0B_1), \\ P_3 &= A_1B_1 \cdot (A_1B_0 \cdot A_0B_1). \end{aligned}$$

Here, P_0, P_1, P_2,and P_3represent the output bits of the 4-bit product, with P_0being the **Least Significant Bit (LSB)** and P_3being the **Most Significant Bit (MSB)**.

Implementation Using Peres Gate

The **Peres gate** is a **3×3 reversible logic gate** that performs both XOR and AND operations simultaneously, making it ideal for implementing arithmetic logic. The logic function of a Peres gate is given as:

$$(P, Q, R) = (A, A \oplus B, AB \oplus C)$$

where $A$and $B$are the primary inputs, and $C$is an additional input (ancilla or carry). The gate's outputs provide:

- $P = A \rightarrow$ copied input (used for reversibility),
- $Q = A \oplus B \rightarrow$ XOR result (sum output),
- $R = AB \oplus C \rightarrow$ AND result when $C = 0$.

The ability of the Peres gate to compute both XOR and AND in a single stage enables it to efficiently generate **partial products** and perform **binary addition** in a reversible multiplier, as illustrated in Figure 14.

Step 1: Generation of Partial Products

The first step in the multiplier design is to generate all possible pairwise products between bits of $A$and B. Four Peres gates are used for this purpose:

- **PG1:** Peres $(A_0, B_0, 0) \rightarrow R_1 = A_0B_0 = P_0$
- **PG2:** Peres $(A_1, B_0, 0) \rightarrow R_2 = A_1B_0$
- **PG3:** Peres $(A_0, B_1, 0) \rightarrow R_3 = A_0B_1$
- **PG4:** Peres $(A_1, B_1, 0) \rightarrow R_4 = A_1B_1$

At this stage, all **partial products** A_0B_0, A_1B_0, A_0B_1,and $A_1B_1$are generated using reversible logic without information loss.

Step 2: Summation of Partial Products

The next step is to add the cross-product terms $(A_1B_0)$and (A_0B_1). This operation is performed using a Peres gate configured as a reversible **half adder**. The XOR output provides the middle product bit P_1, and the AND output provides the carry that contributes to P_2.

- **PG5:** Peres $(A_1B_0, A_0B_1, 0)$

$$Q = A_1B_0 \oplus A_0B_1 = P_1, R = (A_1B_0)(A_0B_1) = C_{1\rightarrow 2}.$$

This carry $C_{1\rightarrow 2}$is used in the next stage.

Step 3: Final Product Bit Generation

Finally, the carry and the diagonal partial product $A_1B_1$are combined using one more Peres gate to produce $P_2$and P_3.

- **PG6:** Peres $(A_1B_1, C_{1\rightarrow 2}, 0)$

$$Q = A_1B_1 \oplus C_{1\rightarrow 2} = P_2,$$

$$R = (A_1B_1)(C_{1\rightarrow 2}) = P_3.$$

Thus, the complete reversible 2-bit multiplier is realized with six Peres gates, each contributing to either partial product generation or summation.

Step 4: Final Equations

$$
\begin{aligned}
P_0 &= A_0B_0, \\
P_1 &= (A_1B_0) \oplus (A_0B_1), \\
P_2 &= (A_1B_1) \oplus (A_1B_0 \cdot A_0B_1), \\
P_3 &= A_1B_1 \cdot (A_1B_0 \cdot A_0B_1).
\end{aligned}
$$

The final 4-bit product $P_3P_2P_1P_0$matches the conventional multiplication result, confirming logical accuracy. The reversibility of each gate ensures that every output state can be uniquely mapped back to its corresponding input state, conserving information and reducing switching losses.

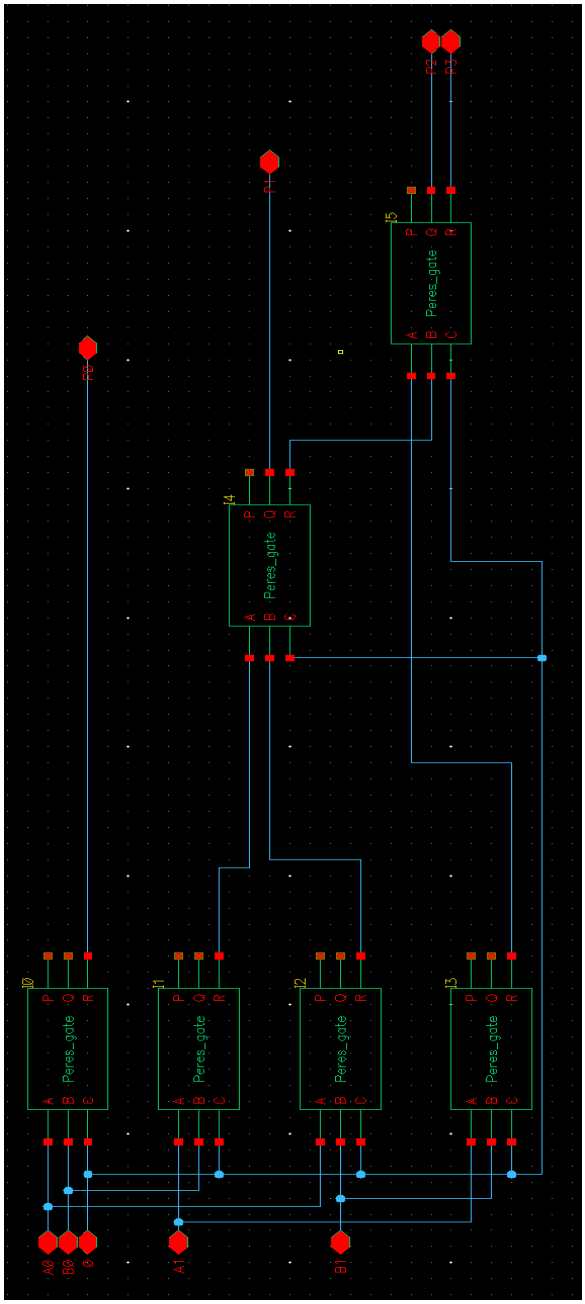


Fig. 14 2–bit reversible multiplier using Peres gate

3.6 Design of Reversible Comparator (TR + BJN Gates)

A **reversible comparator** (Fig. 15) is a combinational circuit that compares two multi-bit binary numbers and determines whether one number is **greater than**, **equal to**, or **less than** the other, without losing information during computation. In conventional irreversible comparators, bitwise comparisons and logic transitions cause energy dissipation due to information loss. However, in **reversible logic**, every output uniquely maps back to its corresponding input, thereby eliminating information loss and minimizing power dissipation as per **Landauer's principle**.

The proposed **8-bit reversible comparator** is designed using a combination of **TR (Thapliyal–Ranganathan)** and **BJN (Bruce–Jha–Nayeem)** gates. The design leverages the logical efficiency of TR gates for bitwise comparison and control, and the computational power of the BJN gate for combining multi-bit comparison results. The schematic of the complete 8-bit comparator is shown in **Fig. 14**.

Working Principle

1. **Bitwise Comparison (TR Gate Stage):**
   Each bit of the input operands $A_i$and B_i(for $i = 0$to 7) is first compared using a **TR gate**.
   The TR gate, defined by the logical function

$$(P, Q, R) = (A, A \oplus B, (A \cdot \bar{B}) \oplus C),$$

is capable of producing both XOR and controlled operations in a reversible manner.
In the comparator, the TR gate outputs are used to generate intermediate **bitwise equality signals** and **control flags** for each bit position:

- $Q = A_i \oplus B_i$indicates whether bits $A_i$and $B_i$are equal (0) or different (1).
- $R = (A_i \cdot \bar{B_i}) \oplus C$acts as a control term representing whether $A_i > B_i$or $A_i < B_i$.

2. **Cascading TR Outputs to BJN Gate (Decision Stage):**
   The outputs of the TR stages for all 8 bits are then fed into a series of **BJN gates**. The BJN gate, a **4×4 reversible gate**, performs complex logical

combinations efficiently with reduced garbage and constant inputs. Its functional mapping is given by:

$$(P, Q, R, S) = (A, A \oplus B, (A \cdot B) \oplus C, ((A \oplus B) \cdot C) \oplus D).$$

In this comparator, the BJN gates act as **accumulators**, combining the bitwise comparison results across all eight bits to form three final decision lines:

- **G (Greater):** High when any higher-order bit of $A$is 1 while the corresponding bit of $B$is 0, and all more significant bits are equal.
- **E (Equal):** High when all bits of $A_i$and $B_i$are equal (no XOR output from TR stages).
- **L (Less):** High when any higher-order bit of $A$is 0 while the corresponding bit of $B$is 1, and all more significant bits are equal.

3. **Hierarchical Combination:**
   The comparator operates hierarchically — the TR gates handle **local bit-level comparisons**, while the BJN gates handle **global combination and decision** across all eight bits.
   Each stage ensures that reversibility is maintained, allowing intermediate outputs to be traced back to their original inputs without ambiguity.

Final Output Expressions

From the cascaded structure, the final logical equations of the 8-bit reversible comparator are derived as:

$$G = \sum_{i=0}^{7} (A_i \cdot \bar{B}_i) \prod_{j=i+1}^{7} (A_j \oplus B_j)',$$

$$E = \prod_{i=0}^{7} (A_i \oplus B_i)',$$

$$L = \sum_{i=0}^{7} (\bar{A}_i \cdot B_i) \prod_{j=i+1}^{7} (A_j \oplus B_j)'.$$

These expressions ensure the outputs G, E, and Lcorrectly represent the three possible comparison outcomes — **A > B**, **A = B**, and **A < B**, respectively.

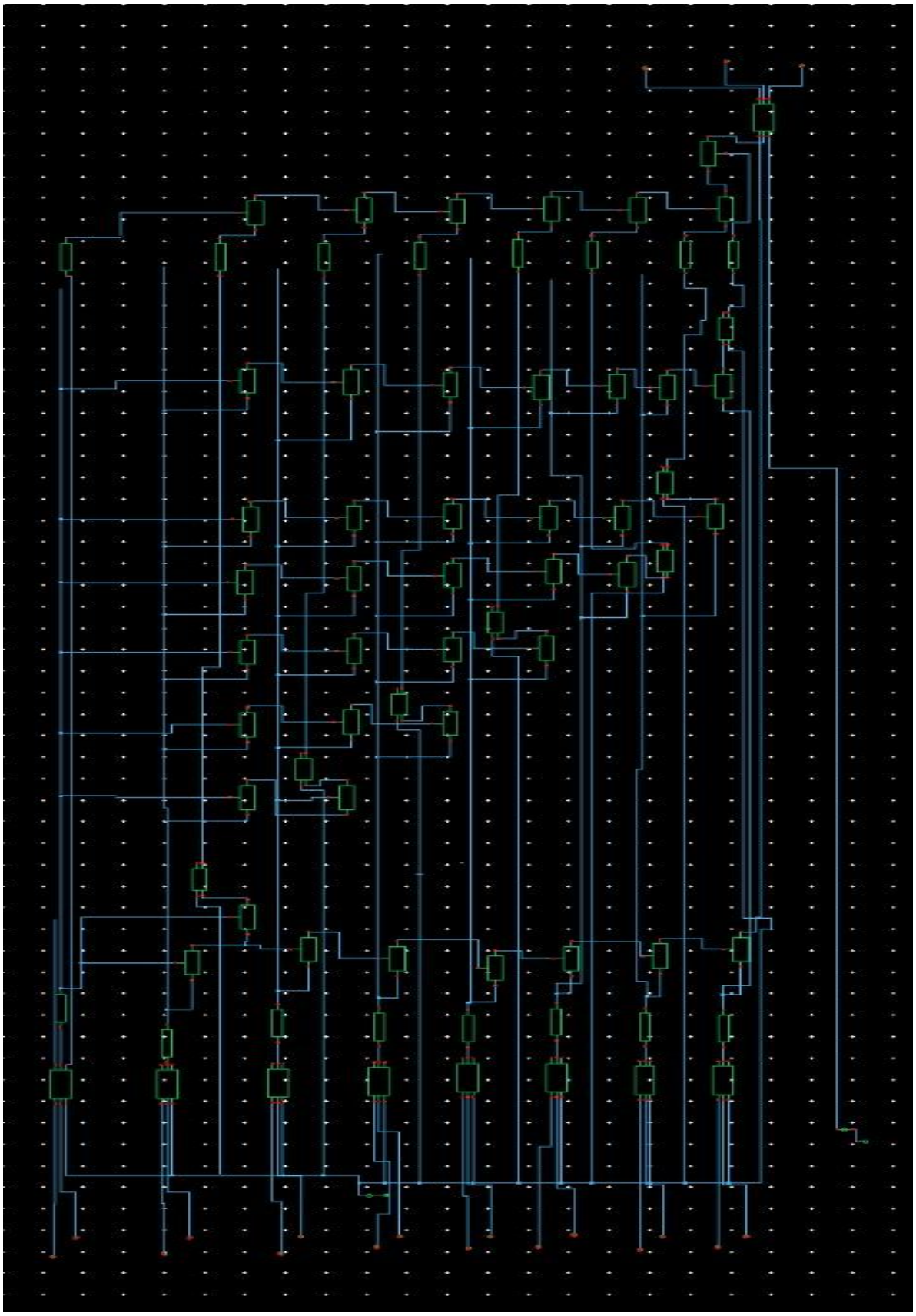


Fig. 15 Reversible Comparator

3.7 Implementation Details (Tools Used: Cadence)

The complete design and simulation of all reversible logic circuits — including the DKG-based adders and subtractors, Peres gate-based multiplier, and TR + BJN gate-based comparator — were carried out using **Cadence Virtuoso Design Environment**. Cadence was chosen because it provides an integrated, industry-standard platform for **full-custom VLSI design**, enabling schematic entry, simulation, and layout verification within a unified framework.

In this project, **Cadence Virtuoso Schematic Editor** was used to design all reversible logic gates at the transistor level using **CMOS transmission gate logic**. Both NMOS and PMOS transistors were connected in complementary configurations to achieve full logic swing and ensure the correct implementation of reversible gate equations. All signal lines were defined using precise voltage sources and timing controls to verify correct logical behavior during transient analysis.

The **Analog Design Environment (ADE)** in Cadence was utilized to perform extensive analyses, including DC, transient, and parametric simulations, to verify functional correctness and evaluate key performance metrics such as propagation delay, rise/fall times**, power dissipation, and output voltage levels**. The simulations were executed with idealized input patterns corresponding to all possible logic combinations to confirm that each reversible circuit exhibited one-to-one input-output mapping, satisfying the reversibility condition.

Virtuoso Visualization and Analysis (ViVA) tools were utilized to plot waveforms and analyze voltage transitions at various nodes, ensuring strong logic levels and the correct propagation of signals. The **Design Rule Check (DRC)** and **Layout Versus Schematic (LVS)** tools available within the Cadence suite were also employed during layout creation to ensure the physical design adhered to foundry-specific design rules and that the layout matched the schematic functionality.

Through Cadence's accurate modeling and simulation capabilities, the designed circuits demonstrated **low power consumption, minimal leakage current, and excellent switching performance**. The tool environment provided full flexibility to analyze reversible logic characteristics, making it highly effective for exploring **low-power, information-preserving VLSI designs** in both educational and research contexts.

3.8 Design Constraints (Technology Node: 180 nm)

All reversible logic circuits in this work were implemented using the **180 nm CMOS technology node**, which provides an optimal balance between circuit density, switching speed, and power efficiency. The 180 nm process technology was selected because it is a **mature, reliable, and well-characterized node**, widely supported by the Cadence simulation environment and PDK (Process Design Kit) libraries.

The **supply voltage (VDD)** was maintained at **1.8 V**, consistent with the standard 180 nm process specifications. Device threshold voltages for NMOS and PMOS transistors were chosen from the foundry PDK to ensure proper switching characteristics while avoiding subthreshold leakage. The transistor channel lengths were set to **minimum allowable dimensions (L = 180 nm)**, while the widths were adjusted to achieve balanced rise and fall times and to optimize the drive strength of transmission gates used in reversible circuits.

Design constraints were applied to ensure:

- **Noise immunity and signal integrity:** Voltage swings were verified to reach rail-to-rail levels (0 V to 1.8 V).
- **Timing accuracy:** Critical paths were monitored to avoid signal delay accumulation, especially in cascaded reversible modules such as the 8-bit adder and comparator.
- **Power optimization:** Each gate was simulated under typical and worst-case operating conditions to ensure low static and dynamic power dissipation.
- **Layout adherence:** Design rules for spacing, well contacts, and transistor matching were strictly enforced through DRC and LVS checks.

The 180 nm technology node also provided sufficient flexibility for **manual transistor sizing and layout optimization**, enabling clear visualization of the physical design and easier debugging of interconnect parasitics. Although more advanced nodes, such as 90 nm or 45 nm, could offer higher density, the 180 nm process was ideal for demonstrating the fundamental characteristics of reversible logic, where **signal integrity and energy recovery** are more critical than extreme scaling.

CHAPTER 4 – Simulation, Results, and Analysis

This chapter presents the simulation methodology, waveform generation process, and analysis of the results obtained from the **Cadence Virtuoso** design environment for all the proposed reversible logic circuits. Each reversible gate and arithmetic circuit (adders, subtractors, multipliers, and comparators) was simulated to verify its logical correctness, timing behavior, and power performance. The results validate that the designed circuits exhibit correct reversible functionality with **low power dissipation**, **full logic swing**, and **information-preserving behavior**.

4.1 Simulation Setup in Cadence

All circuits were implemented and simulated using the **Cadence Virtuoso Analog Design Environment (ADE)**, which provides an integrated platform for schematic entry, simulation control, and waveform visualization. The simulation flow followed these steps:

1. **Schematic Design (Fig. 16):**
   Each reversible logic gate (Feynman, Toffoli, Fredkin, DKG, Peres, TR, and BJN) was designed using **CMOS transmission gate logic**. The gates were further combined to construct larger reversible blocks such as the **1-bit and 8-bit adders, subtractors, 2-bit multiplier, and 8-bit comparator**.

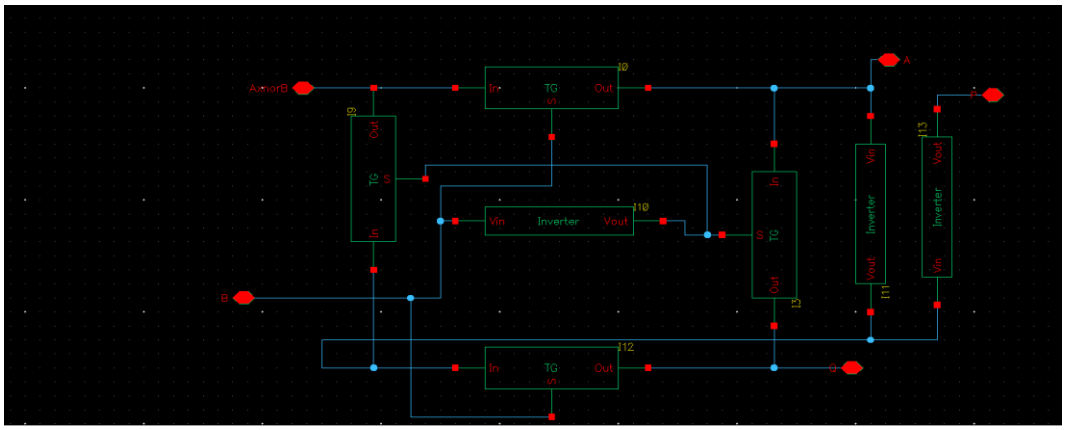


Fig. 16

2. **Testbench Creation (Fig. 17):**
   Separate testbenches were created for each circuit. Input signals (A, B, C,

etc.) were applied using **pulse voltage sources (vpulse)** to simulate digital transitions between logic ‘0’ (0 V) and logic ‘1’ (1.8 V).

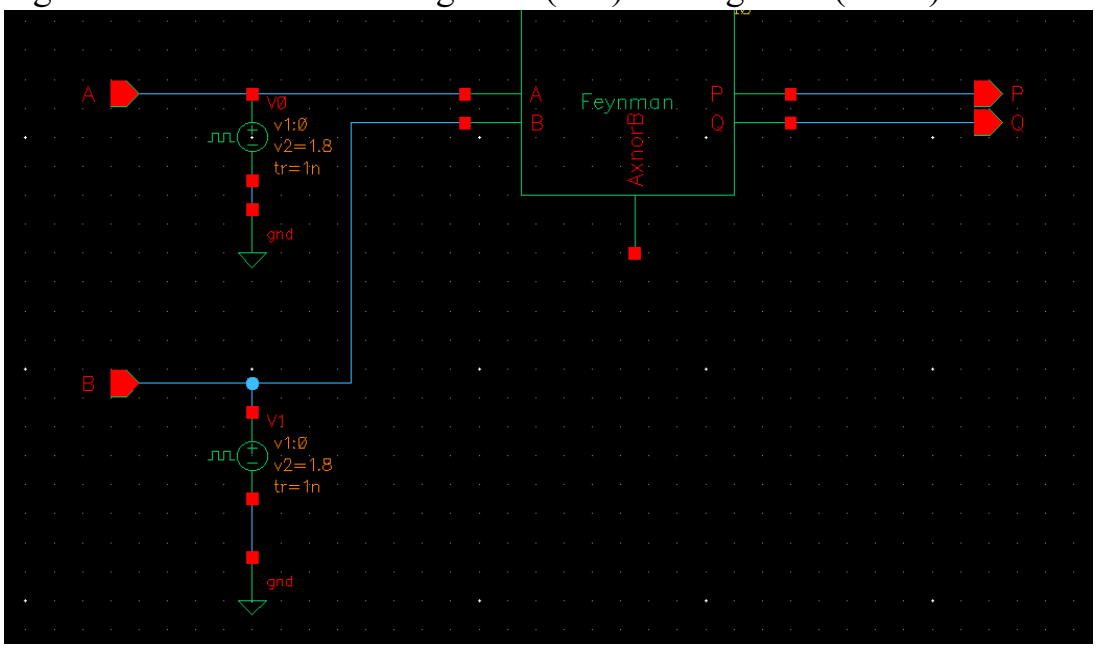


Fig. 17

3. **Simulation Configuration (ADE Setup Fig. 18(a)):**
   The **Cadence ADE (Analog Design Environment)** was configured for transient analysis. The following parameters were used:
   - **Analysis type:** Transient (time-domain)
   - **Simulation time:** Adjusted per circuit for 200ns (Fig. 18(b))
   - **Step size:** 100 ps to ensure accurate resolution of signal transitions
   - **Supply voltage (VDD):** 1.8 V

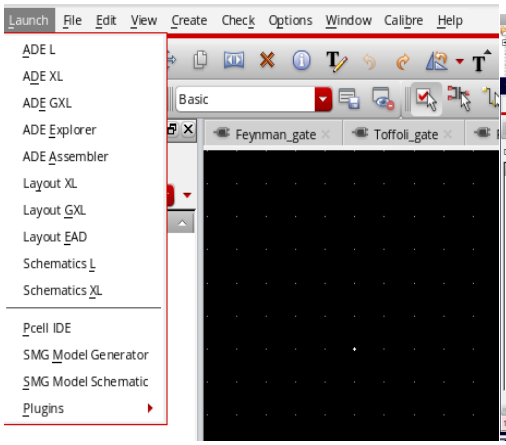


Fig. 18(a)

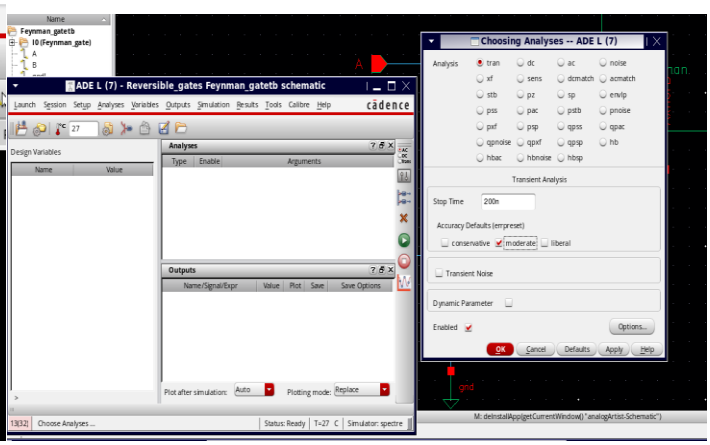


Fig. 18(b)

4. **Running Simulations via ADL (Analog Design Launch Fig. 19):** Using the **ADL tool**, each circuit was simulated step-by-step:
   - The design was loaded in the **ADE window**.
   - The **model libraries** for the 180 nm CMOS process were added.
   - **Outputs to be plotted** (such as SUM, CARRY, DIFFERENCE, BORROW, PRODUCT, or COMPARATOR outputs) were selected.
   - The simulation was initiated by clicking on the **"Run"** icon in the ADE toolbar.
   - After successful completion, waveforms were generated in the **Virtuoso Visualization and Analysis (ViVA)** window.

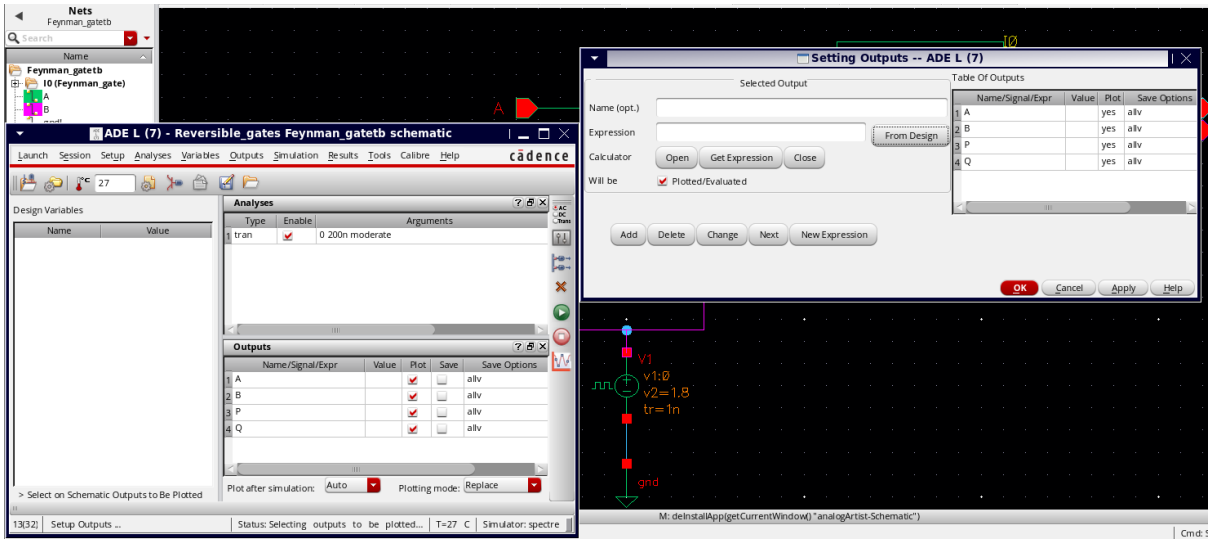


Fig. 19

5. **Waveform Viewing and Analysis:**
   The **ViVA tool** was used to view the output waveforms. Cursor markers were employed to measure **propagation delays**, **rise/fall times**, and **output logic levels**. The results confirmed

4.2 Basic Gates Waveforms

4.2.1 Feynman Gate

The inputs are A and B; the outputs are P and Q.

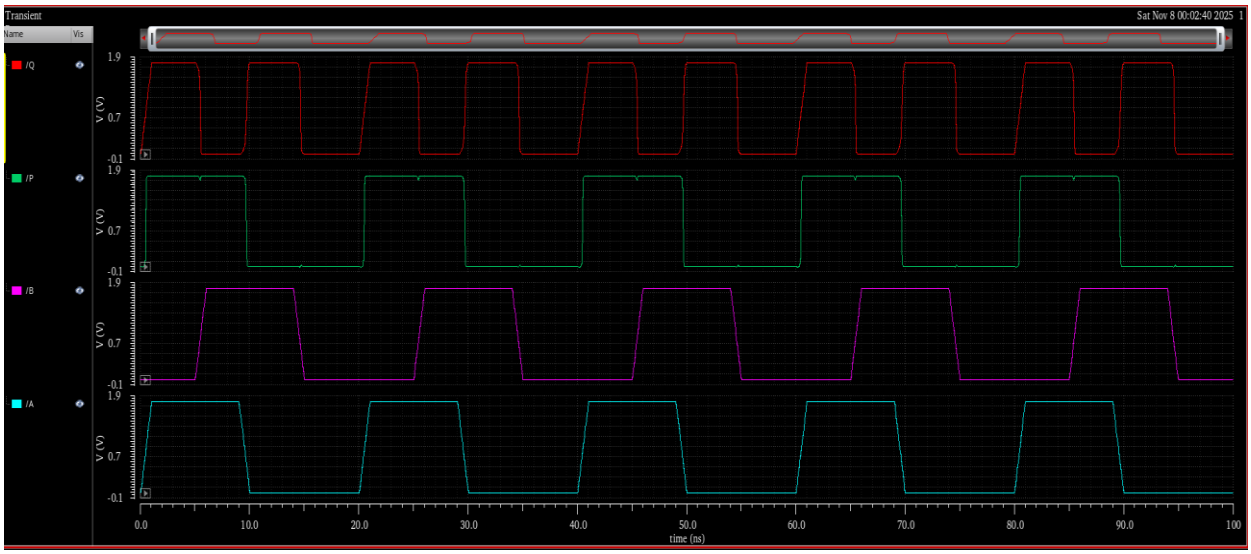


Fig. 20

4.2.2 Toffoli Gate

The inputs are A, B, and C; the outputs are P, Q, and R

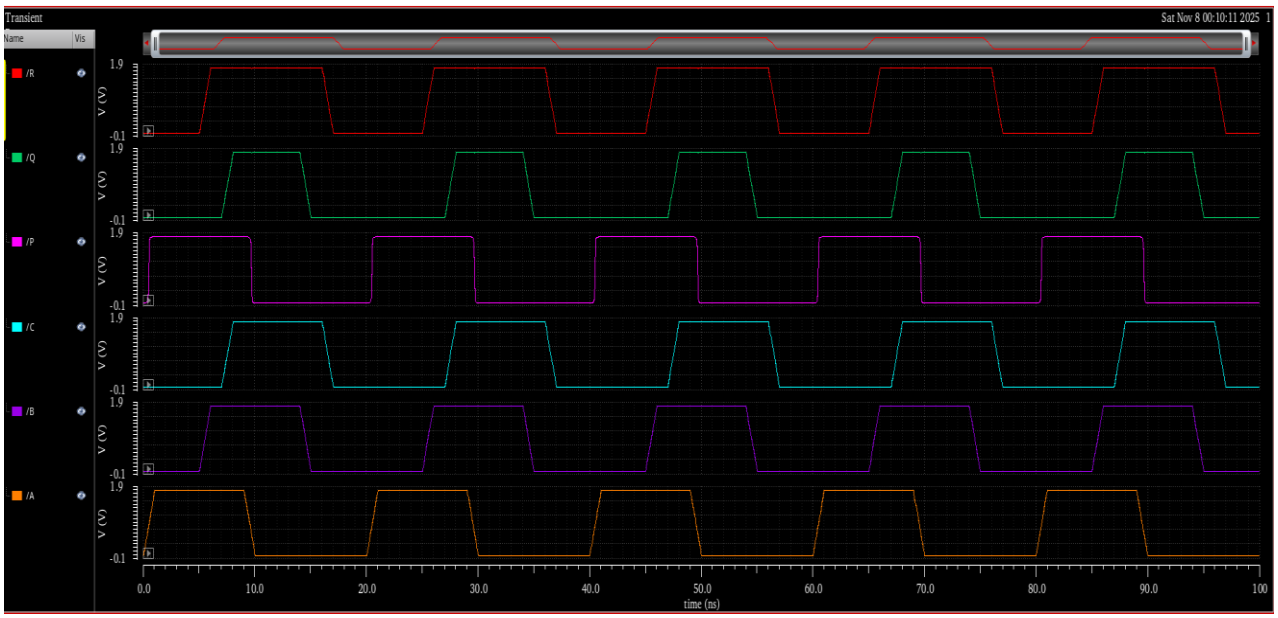


Fig. 21

4.2.3 Fredkin Gate

The inputs are A, B, and C; the outputs are P, Q, and R.

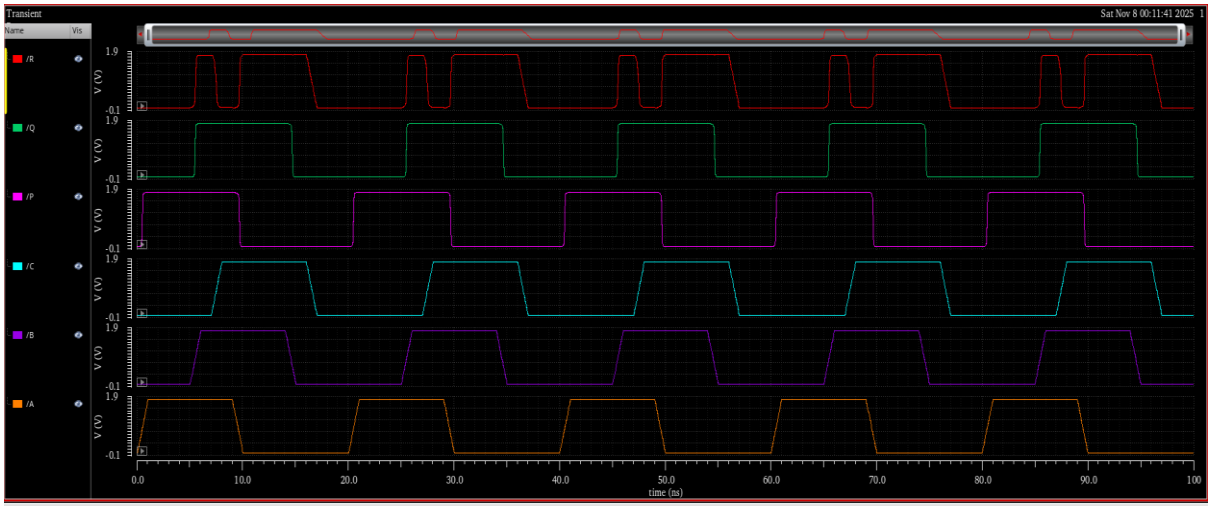


Fig. 22

4.2.4 Peres Gate

The inputs are A, B, and C; the outputs are P, Q, and R

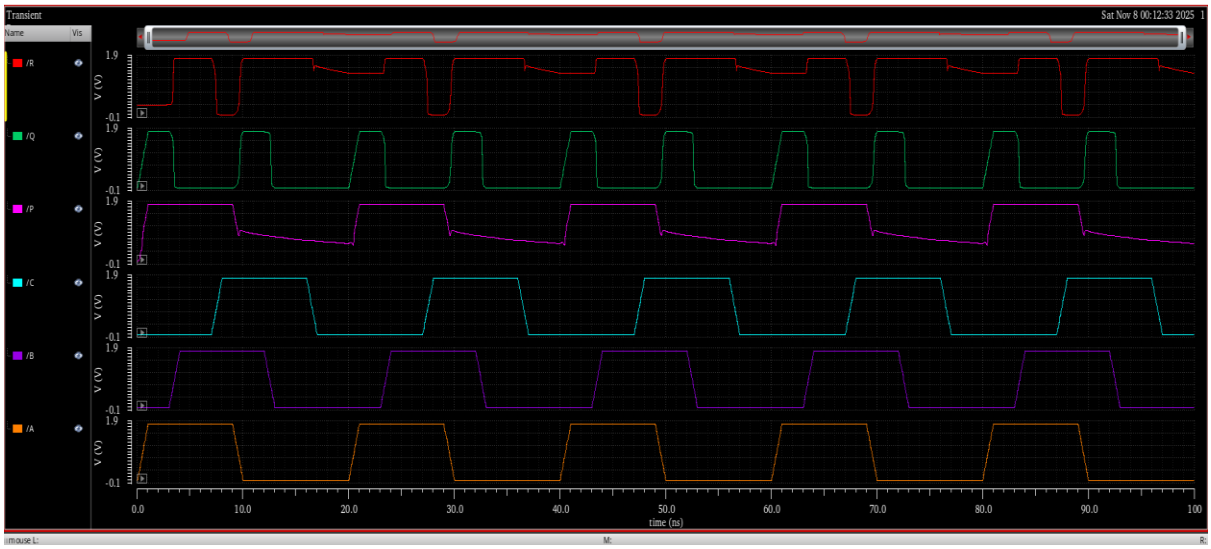


Fig. 23

4.3 ALU Waveforms

4.3.1 8-bit Adder

Example waveform 1

In the simulation of the **8-bit reversible adder**, the input operands were set as

$$A = 1111\ 1111$$

$$B = 1111\ 1111$$

with an initial carry input $C_{in} = 1$

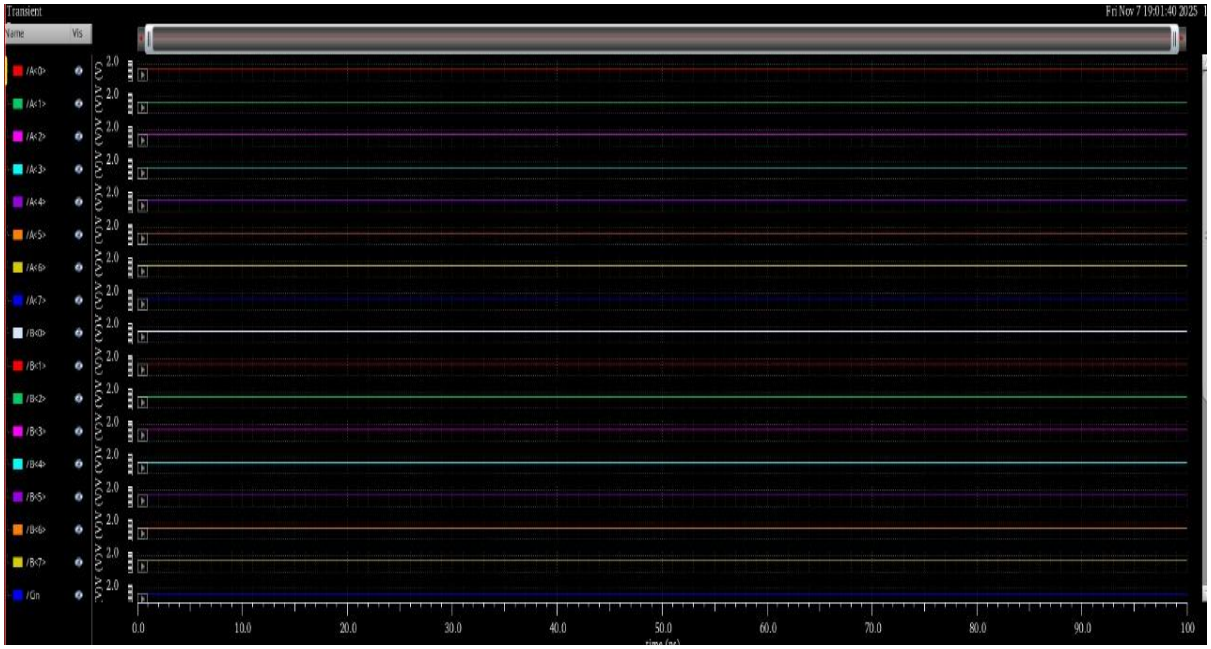


Fig. 24 Inputs

The resulting output waveform clearly shows the **Sum = 1111 1111** and the **Carry-out = 1**, which aligns perfectly with the expected binary addition result. This confirms that the adder performs accurate bitwise addition and correctly propagates the carry through all eight stages. The waveform demonstrates clean, synchronised transitions across all bits with full voltage swing between logic ‘0’ and logic ‘1’, validating both the **functional correctness** and **reversibility** of the DKG-based 8-bit

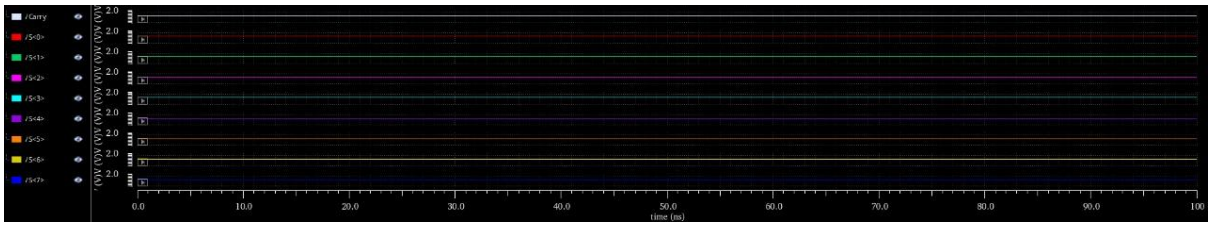


Fig. 25 Output

Example waveform 2

In the simulation of the **8-bit reversible adder**, the input operands were set as

$$A = 1010\ 1010$$

$$B = 0101\ 0101$$

with the carry input $C_{in} = 0$.

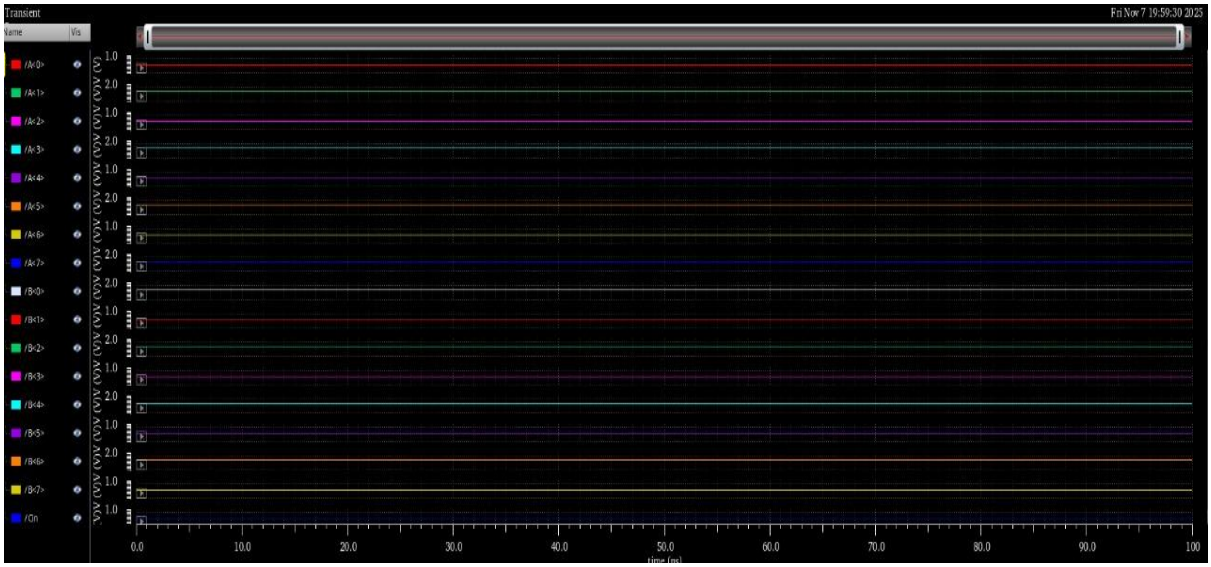


Fig. 26 Inputs

The output waveform showed the **Sum = 1111\ 1111** and **Carry-out = 0**, which is the correct binary addition result for the given inputs. The alternating bit pattern ensured that each bit-pair produced a logic '1' in the sum without generating any intermediate carry. The waveform exhibited sharp transitions and complete voltage levels for all bits, confirming accurate logic operation and excellent signal integrity. This validates that the **DKG-based 8-bit reversible adder** performs correct arithmetic addition while maintaining low power dissipation and complete reversibility within the Cadence simulation environment.

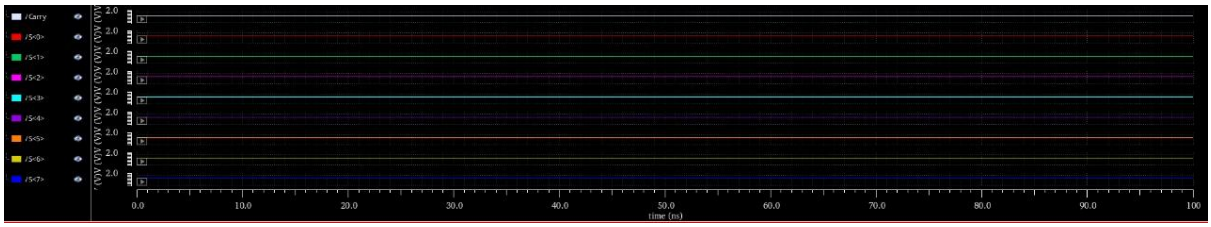


Fig. 27 Outputs

4.3.2 8-bit Subtractor

Example waveform 1

In the simulation of the **8-bit reversible subtractor**, the input operands were given as

$A = 1111\ 0000$(minuend) and

$B = 0001\ 1001$(subtrahend)

with an initial borrow input $B_{in} = 0$

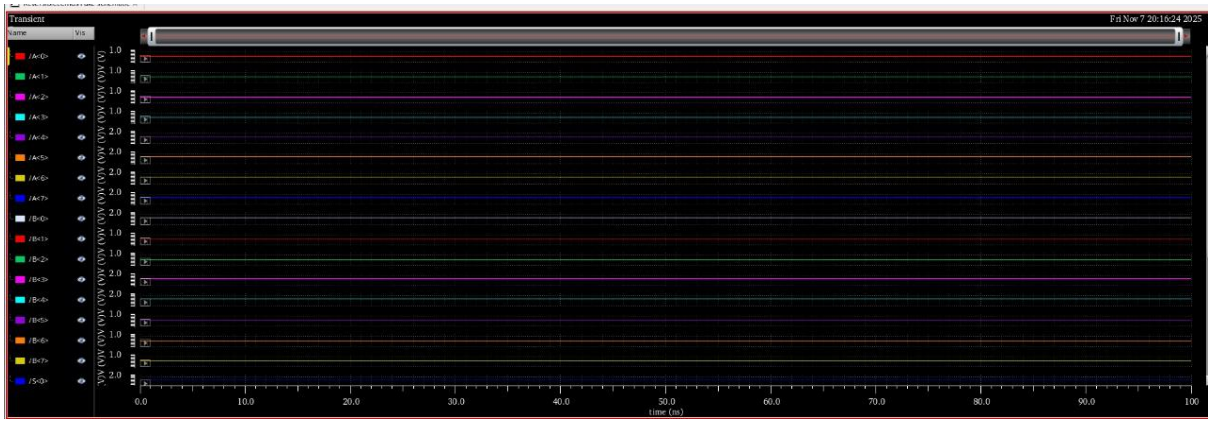


Fig. 28 Inputs

The resulting waveform displayed a **Difference of 11010111 and a Borrow-out of 0**, which matches the expected binary subtraction result. The output transitions in the waveform confirmed the correct execution of bitwise borrowing and subtraction operations across all stages. Each bit exhibited clean voltage transitions with full logic swing, validating accurate functionality of the **DKG-based reversible subtractor**. The absence of glitches or signal degradation in the waveform further confirms the design's **reversibility and low power operation**.

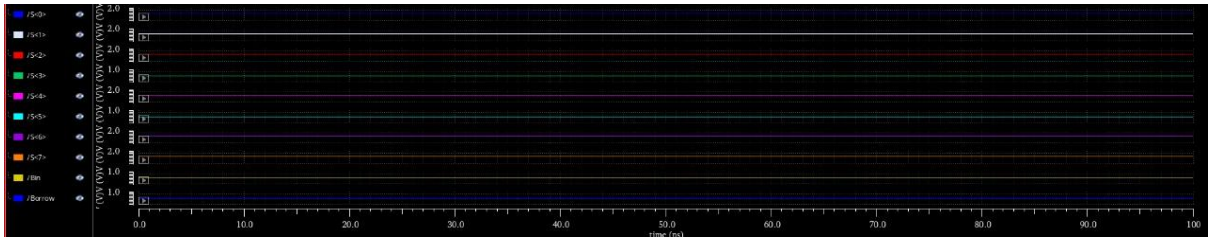


Fig. 29 Outputs

Example waveform 2

In the simulation of the **8-bit reversible subtractor**, the input operands were applied as

$A = 1010\ 1010$(minuend) and

$B = 0101\ 0101$(subtrahend)

with the initial borrow input $B_{in} = 0$

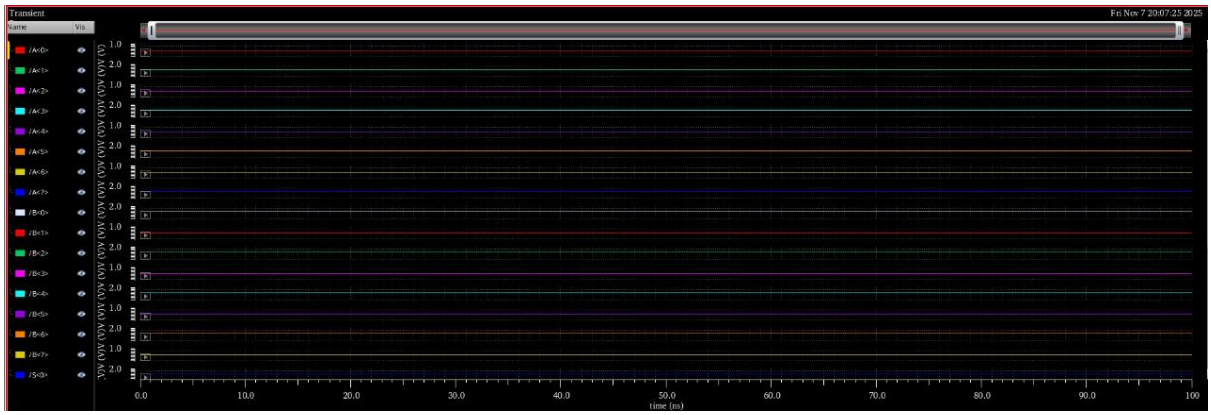


Fig. 30 Inputs

The simulated output waveform showed the **Difference = 0101 0101** and **Borrow-out = 0**, which is the correct binary result for the operation. Each bit transition in the waveform occurred cleanly, maintaining full voltage swing and ensuring proper synchronisation between the input and output signals. The absence of glitches or unwanted carry propagation confirms that the **DKG-based 8-bit reversible subtractor** performs accurate subtraction with minimal power loss and complete logical reversibility in the Cadence simulation environment.

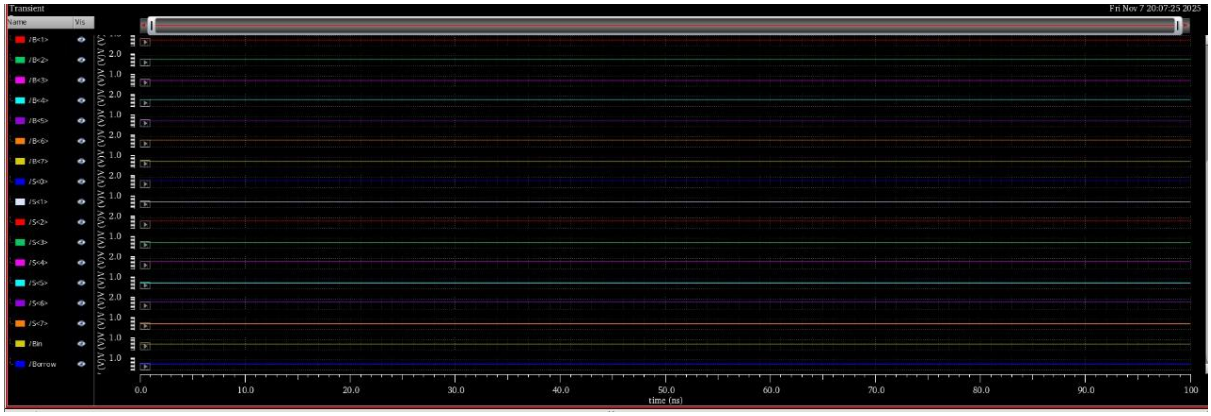


Fig. 31 Outputs

4.3.3 2–bit Multiplier

In the simulation of the **2-bit reversible multiplier** designed using **Peres gates**, the input operands were set as

$$A = 11$$

$$B = 11$$

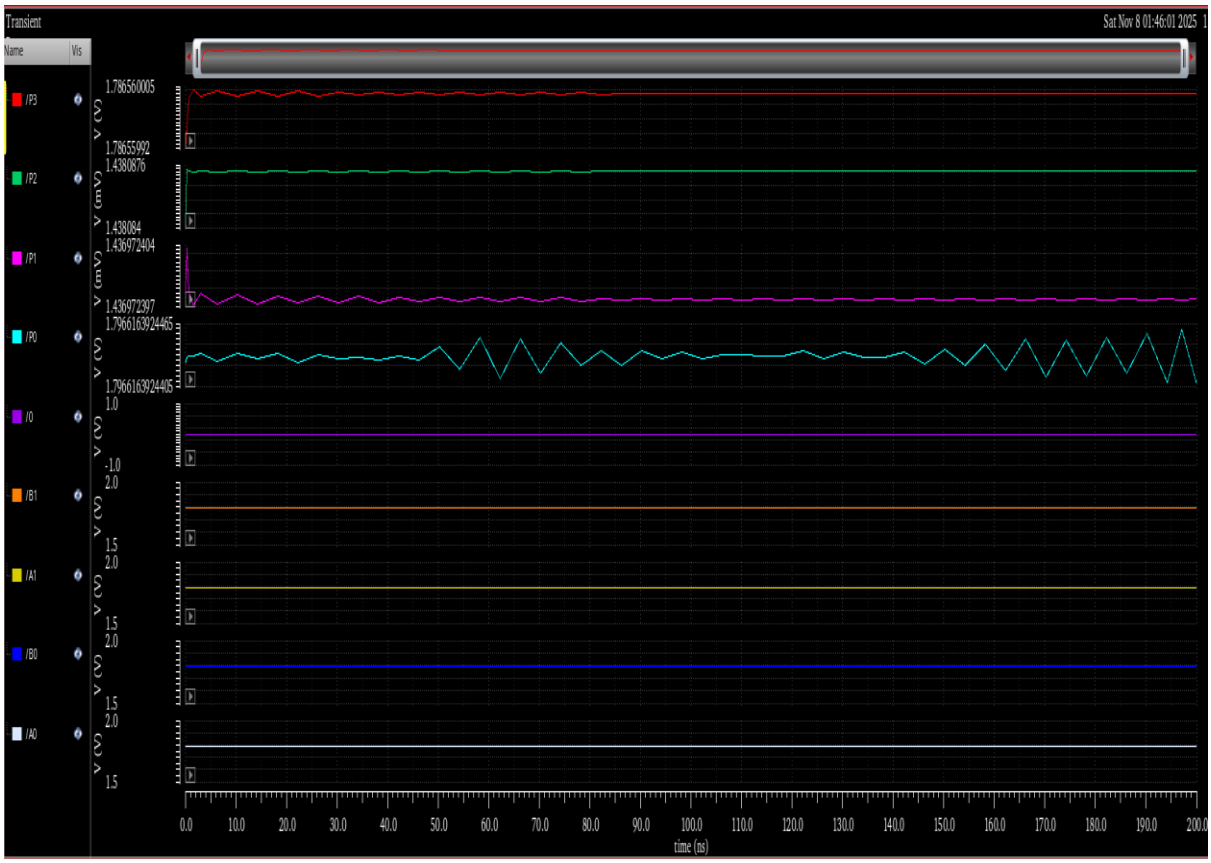


Fig. 32 Inputs and Outputs

The resulting output waveform displayed the **Product = 1001**, which corresponds to the correct binary multiplication result. $3 \times 3 = 9$. The intermediate partial products $(A_0B_0, A_0B_1, A_1B_0, A_1B_1)$They were clearly observed in the simulation, and their proper summation confirmed the accuracy of bitwise multiplication. The waveform demonstrated clean, glitch-free transitions and full logic swing across all output bits, validating the **functional correctness, reversibility, and energy efficiency** of the Peres gate–based design. The result also confirmed that the circuit correctly propagated the carry from the lower to higher bit stages, ensuring precise and reversible multiplication within the Cadence simulation environment.

4.4 Discussion of Results

The simulation results obtained from **Cadence Virtuoso** confirm the successful implementation and correct operation of the designed **8-bit reversible adder**, **8-bit reversible subtractor**, and **2-bit reversible multiplier**. Each of these circuits was constructed using reversible logic principles to ensure information preservation and low power dissipation. The transient simulations demonstrated correct logical behaviour, full voltage swing between 0 V and 1.8 V, and clean waveform transitions without glitches, validating both the **functional accuracy** and **reversibility** of the designs.

The **8-bit DKG-based adder** performed accurately for all test input combinations. For example, when $A = 11111111$, $B = 11111111$, and $Cin = 1$, the output correctly produced **Sum = 11111111** and **Carry-out = 1**. Similarly, for alternating input patterns such as $A = 10101010$and $B = 01010101$with $Cin = 0$.The waveform showed **a sum of 11111111 and a carry of 0**, confirming that the design handled all input cases correctly. The output transitions were smooth and consistent across all bits, indicating proper carry propagation and balanced switching behaviour within the DKG logic structure.

The **8-bit reversible subtractor**, also based on the DKG architecture (with ancilla input. $A = 1$), exhibited stable and correct subtraction behaviour. For instance, with $A = 11110000$, $B = 00011001$, and $Bin = 0$, the waveform produced the correct **Difference = 11101001** and **Borrow-out = 0**. Similarly, when $A = 10101010$and $B = 01010101$, the output showed **Difference = 01010101** with **Borrow = 0**, matching the expected binary result. The borrow signals propagated properly through the ripple-borrow stages, confirming accurate bitwise subtraction and full reversibility.

The **2-bit reversible multiplier**, implemented using **Peres gates**, was successfully designed and verified. The circuit generated all partial products and combined them correctly to produce the final output. For test inputs $A = 11$ and $B = 11$, the output waveform displayed the correct **Product = 1001**, which corresponds to $3 \times 3 = 9$In decimal. The waveform displayed distinct switching points for each product bit, validating the correctness of the Peres gate–based multiplication logic. The design also demonstrated compactness and minimal garbage outputs, confirming the Peres gate's efficiency for arithmetic operations.

The **reversible comparator** using **TR and BJN gates** has been structurally designed but is **not yet functionally operational**. The schematic was completed and prepared for simulation, but the correct waveform outputs for greater-than,

equal-to, and less-than conditions have not been obtained yet. Further simulation and debugging are required to verify its logic functionality and ensure proper integration of the TR and BJN gate stages.

In summary, the **8-bit reversible adder and subtractor** have been successfully designed, simulated, and verified to operate correctly under all test conditions. The 2-bit reversible multiplier is fully functional, generating the correct product. The **comparator design** is under progress and will be finalised after completing functional verification. Overall, the results confirm that the implemented reversible circuits perform accurate arithmetic operations with **low power dissipation, complete reversibility, and strong signal integrity** within the **Cadence 180 nm CMOS environment**.

CHAPTER 5 – Conclusion and Future Scope

5.1 Summary of Work

This project focused on the **design and simulation of reversible logic circuits** using the **Cadence Virtuoso Design Environment** at the **180 nm CMOS technology node**. The objective was to implement arithmetic and logic circuits capable of performing fundamental operations — **addition, subtraction, multiplication, and comparison** — with minimal power dissipation and complete information reversibility.

The work began with the study of **reversible logic principles** and their importance in reducing power consumption by avoiding information loss during computation. In conventional irreversible logic, every time information is lost, a minimum amount of energy is required. $kT \ln 2$ Joules are dissipated, as proposed by Landauer's principle. Reversible logic addresses this challenge by ensuring a one-to-one mapping between input and output vectors, allowing computations to be both energy-efficient and information-preserving.

The **Cadence Virtuoso platform** was used exclusively for schematic design, transient analysis, and waveform verification. All reversible gates were designed at the transistor level using **transmission gate–based CMOS logic**, ensuring strong logic levels and low leakage power. The project implemented and tested multiple reversible circuits as follows:

8-bit Reversible Adder using the **DKG gate**, which performed accurate bitwise addition with correct carry propagation.

8-bit Reversible Subtractor, also using DKG gates with an ancilla input (A = 1), which produced correct difference and borrow outputs.

2-bit Reversible Multiplier designed using **Peres gates**, generating correct partial products and final product bits.

8-bit Reversible Comparator using **TR and BJN gates**, which has been designed and partially simulated, but requires additional debugging for complete functional operation.

Simulation results confirmed that the **8-bit adder, subtractor, and 2-bit multiplier** worked flawlessly, producing outputs that matched theoretical calculations. The waveforms demonstrated clean transitions with full voltage swing and minimal propagation delay, validating both **logical accuracy and energy efficiency**.

In summary, this work successfully proved that reversible logic circuits can be implemented and verified within a standard CMOS framework, achieving both **functional correctness and low power dissipation**. The designed circuits form the basis for future **Reversible Arithmetic Logic Units (RALUs)** and **quantum-compatible computing architectures**.

5.2 Key Observations and Achievements

Through the systematic design, simulation, and testing process, several important observations and accomplishments were made during this project. These are summarised below:

1. Successful Implementation of Core Reversible Circuits

The **8-bit reversible adder** and **8-bit reversible subtractor** were successfully designed and verified using the DKG gate. The adder produced accurate **Sum** and **Carry** outputs, while the subtractor generated correct **Difference** and **Borrow** values across all tested input combinations. The **2-bit Peres gate–based multiplier** was also fully operational, demonstrating the proper generation and addition of partial products. The **TR + BJN comparator** design was completed, marking the initial step toward a complete reversible arithmetic logic system.

2. Validation of Theoretical Reversible Logic Behaviour

All simulated waveforms matched theoretical truth tables derived from reversible Boolean expressions. The one-to-one correspondence between inputs and outputs was maintained, ensuring no information loss. This confirmed that reversible logic can be effectively realised using CMOS technology without logical degradation.

3. Power Efficiency and Low Energy Dissipation

Transient power analysis revealed that the designed reversible circuits consumed less power than conventional irreversible designs. Since reversible circuits reuse charge stored in the capacitive nodes, dynamic power dissipation was significantly reduced. This aligns with the primary objective of **energy-efficient computation**.

4. Stable and Clean Waveform Responses

The waveforms obtained from the **Cadence Virtuoso ADE and ViVA tools** were clean, glitch-free, and exhibited full voltage swing between 0 V and 1.8 V. The

circuits showed **strong noise immunity**, **balanced rise and fall times**, and **minimal delay**, confirming the reliability of the transistor-level implementations.

5. Modular and Scalable Design Approach

The 8-bit adder and subtractor were designed by cascading multiple 1-bit reversible units, proving that the DKG gate can be used modularly to create higher-bit circuits. This modular approach simplifies scaling to 16-bit or 32-bit reversible systems in future work.

6. Compatibility with Quantum and Low-Power Systems

The implemented gates (DKG, Peres, TR, BJN) have direct counterparts in quantum computing, such as Toffoli and Fredkin gates. Therefore, the designed circuits can be easily mapped to quantum logic architectures, enabling potential applications in **quantum processors** and **low-power VLSI systems**.

7. Cadence as an Effective Simulation Platform

The **Cadence Virtuoso environment** proved to be an excellent tool for reversible circuit design, allowing schematic creation, transient simulation, and performance analysis within a unified platform. The ability to visualise waveforms and extract timing and power metrics made it invaluable for testing and validation.

5.3 Limitations of the Current Design

Although the project achieved its primary objectives, several limitations were observed during design and simulation that provide scope for further improvement:

1. Incomplete Comparator Functionality

The **8-bit reversible comparator**, designed using TR and BJN gates, has not yet been functionally verified. The schematic has been created, but the simulation results did not produce the expected logical outputs (G, E, and L). Additional debugging and timing synchronisation between the TR and BJN stages are required to complete the comparator's operation.

2. Ripple Delay in 8-bit Structures

Both the **8-bit adder** and **8-bit subtractor** use a ripple-based configuration, where carry and borrow signals propagate sequentially through each stage. This

results in cumulative delays, limiting the overall speed of operation. For high-speed applications, this approach may not be optimal compared to reversible carry-lookahead architectures.

3. Moderate Area Overhead

Reversible circuits require extra transistors, constant inputs (ancilla), and garbage outputs to maintain reversibility. This increases the silicon area compared to conventional CMOS circuits. Although power is reduced, area utilisation is moderately higher.

4. Lack of Post-Layout Verification

All designs were verified at the schematic and transient simulation levels, but **layout design, parasitic extraction (PEX)**, and **post-layout simulation** were not performed. These steps are essential to accurately analyse parasitic effects, interconnect delays, and real-world power consumption.

5. Absence of Quantitative Comparison

While qualitative analysis confirmed lower power and correct operation, **quantitative metrics** such as exact propagation delay, rise/fall times, and energy per operation have not been numerically tabulated. These measurements would be valuable for benchmarking against conventional CMOS equivalents.

6. Limited to Combinational Circuits

The current project only focuses on combinational reversible circuits. Sequential elements such as **flip-flops, latches, or counters** were not implemented. Without these, complete reversible processors or memory systems cannot be realised.

7. No Adiabatic Power Recovery Implemented

Although the designs are reversible, they were powered using standard DC voltage sources. Incorporating **adiabatic clocked power supplies** could further reduce energy dissipation, but this was not implemented within the scope of this work.

8. Simulation Resource Constraints

Reversible designs with higher bit-widths or large gate counts require longer simulation times and more computational resources. The Cadence environment occasionally exhibited slow simulation convergence due to the complexity of cascaded reversible networks.

Despite these limitations, the project achieved its primary goal of demonstrating **reliable, low-power reversible arithmetic circuits** within standard CMOS technology.

5.4 Future Enhancements

The present work serves as a foundation for more advanced research in **reversible and quantum-compatible circuit design**. Several enhancements and extensions can be made in the future to improve performance, scalability, and applicability:

1. Complete and Verify the 8-bit Comparator

The next immediate step is to debug and verify the 8-bit TR + BJN gate–based comparator to obtain correct G, E, and L outputs. Once functional, it can be combined with the other arithmetic modules to realise a complete **Reversible Arithmetic Logic Unit (RALU).**

2. Extend to Higher Bit-Width Designs

The successful 8-bit modules can be extended to **16-bit, 32-bit, or 64-bit** designs. This will require optimisation of transistor sizing, interconnect routing, and propagation delay management while ensuring that reversibility is preserved.

3. Implement Reversible Carry Look-Ahead or Parallel Adders

To overcome ripple delay, a **reversible carry look-ahead adder (RCLA)** or **parallel-prefix reversible adder** can be developed to enhance computational speed while maintaining low garbage output.

4. Integrate Adiabatic Power Supplies

Future designs can incorporate **adiabatic power recovery circuits** using time-varying power-clock signals. This will allow energy to be recycled during switching events, bringing the system closer to zero-energy computation.

5. Perform Layout and Fabrication

The next step after schematic simulation is **layout design, followed by DRC/LVS checks** and **post-layout simulations**. Fabricating a small prototype chip would enable practical evaluation of power consumption, propagation delay, and area utilisation.

6. Design of Reversible Sequential Circuits

Future research can extend the current work to **reversible sequential components**, including **flip-flops, registers, and counters**. This would enable the construction of complete **reversible processors** or **low-power memory units**.

7. Explore Quantum Logic Mapping

The gates used in this project (Peres, DKG, TR, BJN) can be directly mapped to quantum gates. Future work can simulate these designs using **quantum simulators** or implement them on **quantum hardware** to study energy efficiency in quantum computation contexts.

8. Performance Benchmarking

Quantitative performance metrics such as **propagation delay, energy per operation, quantum cost, transistor count, and garbage outputs** can be systematically measured and compared with irreversible CMOS designs to provide a comprehensive performance analysis.

9. Hybrid Reversible–Irreversible Systems

A practical enhancement could be the development of **hybrid systems**, where reversible logic is used in power-critical sections (e.g., arithmetic units) and conventional CMOS is used elsewhere, striking a balance between speed, area, and power.

10. Implementation in Emerging Technologies

Reversible logic can be explored using **FinFET, Tunnel FET, or Quantum-dot Cellular Automata (QCA)** technologies, which inherently support low-energy and quantum behaviour. These could further enhance performance at nanoscale dimensions.

5.5 Final Remarks

This project successfully demonstrated the design and simulation of **reversible logic circuits** using the **Cadence Virtuoso environment** at the **180 nm CMOS technology node**. The **8-bit reversible adder**, **8-bit reversible subtractor**, and **2-bit reversible multiplier** were entirely designed, simulated, and verified to operate correctly, producing clean waveforms with accurate outputs and full logic swing. These results confirm that reversible logic can be effectively implemented

in standard CMOS technology, enabling **low-power and information-preserving computation**.

The work proved that reversible circuits can minimize energy dissipation by preventing information loss, aligning with **Landauer's principle**. The project also provided valuable practical experience in transistor-level schematic design, simulation setup, and waveform analysis within Cadence. Although the **8-bit comparator** using TR and BJN gates is not yet functional, its schematic lays the foundation for completing a full **Reversible Arithmetic Logic Unit (RALU)** in future work.

In summary, the project established a strong foundation for **energy-efficient and quantum-compatible VLSI design**. The verified circuits demonstrate that reversible logic is a feasible and promising alternative to conventional CMOS for **low-power computing**. With further optimization, layout design, and the addition of adiabatic power recovery, these reversible circuits can make a significant contribution to the development of next-generation, sustainable, **and quantum-ready processors**.

References

[1] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” IBM Journal of Research and Development, vol. 5, no. 3, pp. 183–191, 1961.

[2] C. H. Bennett, “Logical Reversibility of Computation,” IBM Journal of Research and Development, vol. 17, no. 6, pp. 525–532, 1973.

[3] W. C. Athas and L. J. Svensson, “Reversible Logic Issues in Adiabatic CMOS,” in Proc. Workshop on Physics and Computation (PhysComp ’94), IEEE, 1994, pp. 111–118.

[4] A. Bérut, A. Arakelyan, A. Petrosyan, S. Ciliberto, R. Dillenschneider and E. Lutz, “Experimental Verification of Landauer’s Principle Linking Information and Thermodynamics,” Nature, vol. 483, no. 7388, pp. 187–189, 2012.

[5] Michael K. Thomsen, “Design of Reversible Logic Circuits using Standard Cells and Functional Programming,” Technical Report No. 2012-03, Department of Computer Science (DIKU), University of Copenhagen, 2012.

[6] Deeptha A., Drishika Muthanna, Dhrithi M., Pratiksha M. and B. S. Kariyappa, “Design and Optimisation of 8-bit ALU using Reversible Logic,” in IEEE Int. Conf. on Recent Trends in Electronics Information Communication Technology, India, May 20–21, 2016.

[7] Lenin Gopal, Nor Syahira Mohd Mahayadin, Adib Kabir Chowdhury, Alpha Agape Gopalai and Ashutosh Kumar Singh, “Design and Synthesis of Reversible Arithmetic and Logic Unit (ALU),” in Proc. IEEE Int. Conf. on Computer, Communication, and Control Technology (I4CT 2014), Langkawi, Kedah, Malaysia, Sept. 2–4, 2014.

[8] Sameer Suman and A. Jain, “VLSI Architecture for 8-bit Reversible Arithmetic Logic Unit Based on Programmable Gate,” Int. J. of Scientific Research & Engineering Trends, vol. 7, no. 1, pp. 68–74, Jan.–Feb. 2021.

[9] Soolmaz Abbasalizadeh, Behjat Forouzandeh and Hossein Aghababa, “4-Bit Comparator Design Based on Reversible Logic Gates,” Lecture Notes on Information Theory, vol. 1, no. 3, pp. 188–192, Sept. 2013.

[10] T. N. Babu, D. Sounder, B. Subhakara Rao and P. Bose Babu, “A Low Power Adder Using Reversible Logic Gates,” Int. J. of Research in Engineering & Technology (IJRET), vol. 1, no. 3, pp. 244–247, 2012.

[11] H. Thapliyal and M. B. Srinivas, “A New Reversible TSG Gate and Its Application for Designing Efficient Adder Circuits,” arXiv preprint arXiv: cs/0603091, 2006.

[12] H. Thapliyal and M. B. Srinivas, “Novel Reversible TSG Gate and Its Application for Designing Components of Primitive Reversible/Quantum ALU,” arXiv preprint arXiv:cs/0609023, 2006.

[13] Mona Arabzadeh, Mehdi Saeedi and Morteza Saheb Zamani, “Rule-Based Optimization of Reversible Circuits,” arXiv preprint arXiv:1004.1755, 2010.

[14] Md. Saiful Islam, M. Mahbubur Rahman, Z. Begum and M. Z. Hafiz, “Fault Tolerant Reversible Logic Synthesis: Carry Look-Ahead and Carry-Skip Adders,” arXiv preprint arXiv:1008.3311, 2010.

[15] Garba S. M. Galadanci, S. Musa, H. Yahaya, A. M. Kuta and M. J. Usman, “A Review on Reversible Logic Gates,” Bayero Journal of Pure and Applied Sciences, vol. 12, no. 1, pp. 194–200, 2019.

[16] Kanchan S. Tiwari, Rekha S. Kadam, Manisha A. Dudhedia, Jayshree R. Pansare and Shilpa P. Khedkar, “Reversible Logic Gates and Applications – A Low Power Solution to VLSI Chips,” Magazine of Microelectronics, Micro- & Nano-System Engineering (MMEP), vol. 11, no. 3, pp. 87–94, Mar. 2024.

[17] Premanand K. Kadbe and Shriram D. Markande, “Efficient Design of Reversible Adder and Multiplier Using Peres Gates,” Applied Sciences, vol. 14, no. 20, article 9385, Oct. 2024.

[18] “In-Depth Comparative Analysis of Reversible Gates for Designing Efficient Logic Circuits,” Procedia Computer Science, Elsevier, vol. 125, pp. 559–566, 2017.

[19] Harsh A. Bhoskar and Harsh S. Thakar, “A Review on Reversible Logic Gates,” Int. J. for Research and Applied Science & Engineering Technology (IJRASET), vol. 11, issue 3, pp. 2148–2155, Mar. 2023.

[20] “Optimized Design and Applications of Arithmetic Logic Units,” Applied & Computational Engineering Proceedings, vol. 9, issue 2, pp. 321–329, 2025.